

TRABAJO FIN DE GRADO



UCAM

UNIVERSIDAD CATÓLICA
DE MURCIA

ESCUELA POLITÉCNICA SUPERIOR

Grado en Ingeniería Informática

OPTIMIZACIÓN DEL TRÁFICO EN LOS CRUCES A TRAVÉS DE LA INTELIGENCIA ARTIFICIAL

Autor/a:

Jose Ángel Gallego García

Director/a:

Dr. Francisco Arcas Túnez

Murcia, Junio de 2024

TRABAJO FIN DE GRADO



UCAM

UNIVERSIDAD CATÓLICA
DE MURCIA

ESCUELA POLITÉCNICA SUPERIOR

Grado en Ingeniería Informática

OPTIMIZACIÓN DEL TRÁFICO EN LOS CRUCES A TRAVÉS DE LA INTELIGENCIA ARTIFICIAL

Autor/a:

Jose Ángel Gallego García

Director/a:

Dr. Francisco Arcas Túnez

Murcia, Junio de 2024

Agradecimientos

Quiero dar las gracias por el apoyo recibido para el desarrollo de este proyecto, a mis abuelos, a mis padres y a mi novia porque han sido un pilar fundamental a lo largo de toda la carrera y siempre han creído en mí. Respecto al profesorado quiero hacer una mención a mi tutor ya que ha sido una persona comprensible y resolutiva, que me ha sabido guiar de una forma muy correcta.

Listado de Abreviaturas

API, Application Programming Interface

CRUD, Create, Read, Update, Delete

DL, Deep Learning

IA, Inteligencia Artificial

ML, Machine Learning

SUMO, *Simulation of Urban Mobility*

YOLOv8, *You Only Look Once version 8*

XML, *eXtensible Markup Language*

GPU, *Graphic Processing Unit*

ÍNDICE

RESUMEN	16
1. INTRODUCCIÓN	19
1.1. Motivación.....	20
1.2. Definición	20
1.3. Objetivos propuestos (generales y específicos)	21
1.3.1. Objetivos generales	22
1.3.2. Objetivos específicos	22
2. ESTUDIO DEL MERCADO.....	23
2.1. Conceptos relevantes del dominio de aplicación.....	23
2.2. Relación con proyectos con la misma funcionalidad	25
2.3. Estudio de viabilidad	29
2.3.1. Alcance del proyecto	29
2.3.2. Estudio de la situación actual	30
2.3.3. Estudio y valoración de las alternativas de solución	31
2.3.4. Selección de la solución	31
3. METODOLOGÍAS USADAS.....	33
3.1. Metodología Tradicional.....	33
3.1.1. Metodología en cascada	34
3.1.2. Metodología RUP	36
3.2. Metodología ágil.....	36
3.3. Metodología elegida.....	37
4. TECNOLOGÍAS Y HERRAMIENTAS UTILIZADAS EN EL PROYECTO	40
4.1. Tecnologías	40
4.1.1. YOLOv8	40
4.1.2. TraCI	41
4.1.3. PyTorch.....	41

4.2.	Herramientas de desarrollo	41
4.2.1.	Visual Studio Code.....	42
4.2.2.	Python.....	42
4.2.3.	Netedit	42
4.2.4.	SUMO	43
4.2.5.	Irium Webcam.....	43
4.3.	Equipo Hardware	43
4.3.1.	Ordenador de sobremesa personalizado	44
4.3.2.	Nothing Phone 1	44
4.4.	Software de ayuda.....	44
4.4.1.	Git	44
4.4.2.	GitHub.....	45
5.	ESTIMACIÓN DE RECURSOS Y PLANIFICACIÓN.....	47
5.1.	Sprint 0	47
5.1.1.	Planificación del Sprint.....	47
5.2.	Especificación de requisitos	48
5.3.	Planificación del Product Backlog	50
5.4.	Planificación temporal.....	56
5.5.	Estimación de costes	58
6.	DESARROLLO DEL PROYECTO	61
6.1.	Sprint 1	61
6.1.1.	Planificación del Sprint.....	62
6.1.2.	Retrospectiva del Sprint	63
6.2.	Sprint 2	63
6.2.1.	Planificación del Sprint.....	64
6.2.2.	Retrospectiva del Sprint	66
6.3.	Sprint 3	67

6.3.1. Planificación del Sprint	68
6.3.2. Planificación del Sprint	69
6.4. Sprint 4	70
6.4.1. Planificación del Sprint	71
6.4.2. Retrospectiva del Sprint	72
6.5. Sprint 5	72
6.5.1. Planificación del Sprint	73
6.5.2. Retrospectiva del Sprint	73
7. DESPLIEGUE Y PRUEBA DE LA SOLUCIÓN	75
7.1. Plan de pruebas.....	75
7.2. Escalabilidad, extensión, mantenimiento y soporte	85
8. CONCLUSIONES	87
8.1. Objetivos alcanzados	87
8.2. Conclusiones del trabajo y personales	88
8.3. Vías futuras.....	89
9. BIBLIOGRAFÍA	91

ÍNDICE DE ELEMENTOS GRÁFICOS

FIGURAS

Figura 1. Esquema del funcionamiento de la metodología en cascada	35
Figura 2. Requisitos generales	50
Figura 3. Red de carretera	65
Figura 4. Etiquetado de un coche	66
Figura 5. Estadísticas de entrenamiento	68
Figura 6. Detección por regiones.....	69

TABLAS

Tabla 1. Historia de usuarios	50
Tabla 2. Product Backlog.....	53
Tabla 3. Horario de trabajo	57
Tabla 4. Estructura temporal del Sprint.....	58
Tabla 5. Descripción Sprint 1.....	61
Tabla 6. Descripción Sprint 2.....	63
Tabla 7. Descripción Sprint 3.....	67
Tabla 8. Descripción Sprint 4.....	70
Tabla 9. Descripción Sprint 5.....	72
Tabla 10. Descripción del plan de pruebas	75
Tabla 11. Especificación de los casos de prueba	78

RESUMEN

La idea abordada, en este Trabajo de Fin de Grado (TFG), es la congestión en los cruces de carreteras, un desafío que afecta a la movilidad y calidad de la circulación, tanto dentro, como fuera de las ciudades. El objetivo principal, es el diseño de una aplicación, basada en Inteligencia Artificial (IA), la cual, gestione y mejore la fluidez del tráfico, utilizando datos en tiempo real, y así poder reducir los tiempos de espera en los semáforos. Para ello, se ha empleado un enfoque, que combina algoritmos de aprendizaje automático, con técnicas de detección en tiempo real, mediante el uso de cámaras. Los resultados obtenidos son procesados, para poder ajustar los tiempos de los semáforos de manera dinámica, y llevados a un simulador de circulación, el cual, aplica los cambios de manera instantánea, como si de la vida real se tratase. Los resultados obtenidos dejan ver una reducción significativa en los tiempos de espera y una mejora en la eficiencia del tráfico, respecto a los sistemas convencionales utilizados. En síntesis, el uso de inteligencia artificial y detección en tiempo real, demuestran ser soluciones eficaces para optimizar la circulación en los cruces de carreteras.

Palabras claves:

Inteligencia artificial, tráfico, simulación, detección en tiempo real, scrum

ABSTRACT

The idea addressed, in this Final Degree Project (TFG), is congestion at road intersections, a challenge that affects mobility and quality of circulation, both inside and outside cities. The main objective is the design of an application, based on Artificial Intelligence (AI), which manages and improves traffic flow, using real-time data, and thus reduces waiting times at traffic lights. To achieve this, an approach has been used that combines machine learning algorithms with real-time detection techniques using cameras. The results obtained are processed, to be able to adjust the traffic light times dynamically, and taken to a traffic simulator, which applies the changes instantly, as if it were real life. The results obtained show a significant reduction in waiting times and an improvement in traffic efficiency, compared to the conventional systems used. In summary, the use of artificial intelligence and real-time detection prove to be effective solutions to optimize circulation at road intersections.

Keywords:

Artificial intelligence, traffic, simulation, real-time detection, scrum

1. INTRODUCCIÓN

Es incuestionable el hecho de la evolución constante de las tecnologías desde hace mucho tiempo, hasta el punto de la integración total en nuestro día a día, no solo con dispositivos específicos, como puede ser nuestro teléfono móvil o nuestro ordenador, si no con otro tipo de aparatos, que, por ciencia infusa, dábamos por hecho que no tendrían nada que ver con una evolución tecnológica tan grande como la que tienen hoy, como un reloj o la aspiradora.

Hasta hace unos años resultaba imposible, para el usuario común, pensar que una máquina iba a poder, no solo pensar por ella misma, si no también aprender, mejorar y evolucionar, y es que es raro hoy en día no conocer un producto al que se le haya agregado la Inteligencia Artificial (IA) para mejorar su uso, su funcionamiento y su alcance en la tarea designada.

No es difícil encontrar diversos ejemplos donde el avance tecnológico es primordial e indispensable para un mejor desarrollo y funcionamiento de ese ámbito, es por lo que, en diversos campos como la medicina o la industria han desembocado en una evolución rápida y constante, que nos han permitido elevar la calidad de vida a niveles exponenciales en muy poco tiempo.

Sin embargo, dentro de nuestra civilización tecnológica hay campos que todavía no han sido mejorados por diversos motivos, entre estos campos se encuentra el protagonista de este Trabajo de Fin de Grado (TFG), la seguridad vial. A pesar de disponer de una infinidad de normas y medios que garantizan más seguridad en la carretera, es un detalle para comentar, que está a penas integrada con la tecnología.

Con la llegada de la Inteligencia Artificial (IA) se abrió un mundo de posibilidades, en todos los campos de la ciencia, pero hasta el día de hoy, al menos en España, todavía no hemos combinado estos dos elementos que pueden llegar a encajar tan bien. En este trabajo, se propone un ejemplo de fusión entre ambos elementos, que no solo optimizaría la circulación en carretera, si no que aumentaría la seguridad en ella.

1.1. Motivación

Cuando se habla de circulación en carretera, especialmente en cruces, por todos es sabido que hay situaciones especialmente desagradables. Como cuando debemos esperar que un semáforo, en el que no hay nadie al otro lado cruzando, se ponga en verde; semáforos que tardan demasiado en dar paso a otros coches; semáforos que, a altas horas de la noche, quedan todo el rato en ámbar, sin hacer regulación de ningún tipo, o aquellos que provocan atascos innecesarios.

En cualquiera de estos casos el sistema de regulación de la circulación se limita a una cuenta atrás, que cuando llega a su fin, cambia el color y da preferencia de circulación de la vía, sin tener en cuenta la demanda de esta. Por estos motivos, nace la necesidad de este proyecto, generar un sistema inteligente que sea capaz de leer la demanda del tráfico en tiempo real y genere un sistema de preferencia de paso en la vía.

De esta forma, la realización de este proyecto permite llevar a la práctica, ciertos conocimientos adquiridos a lo largo de este grado, y el desarrollo, e investigación, de otros conocimientos que son de interés propio.

En definitiva, lo que me ha llevado a poder desarrollarlo de manera dinámica, efectiva y con una gran motivación, es poder observar como he podido poner en práctica todos los conocimientos adquiridos a lo largo de estos años de carrera. Además, ver cómo con nuestro trabajo podemos llegar a una solución para un problema, es una sensación muy gratificante.

1.2. Definición

Para este proyecto se ha desarrollado una aplicación, en Python, que es capaz de leer el tráfico en tiempo real, haciendo uso de inteligencia artificial.

Se ha entrenado un modelo personalizado, que recoge la información de la circulación y tráfico, centrándonos en aquellos cruces que poseen un semáforo, a través una cámara de vídeo, y en base a los datos detectados, genera un sistema de prioridades de paso.

Para que todo esto sea posible, se ha creado una simulación a través de *software*, que ejecuta un ejemplo de flujo de circulación, el cual, es leído por la cámara del móvil.

Al mismo tiempo es ejecutado el programa que contiene la inteligencia artificial y recoge los datos capturados a través de la cámara del teléfono, dichos datos son transferidos a través de una Application Programming Interface (API), y procesados por un sistema que define el flujo del tráfico, en base a umbrales de cantidad y prioridad.

Una vez que el programa ha determinado las prioridades de paso, se reajustan los parámetros de la simulación, y de nuevo son enviados a través de la API, para que sean aplicados dentro del simulador.

1.3. Objetivos propuestos (generales y específicos)

Con el desarrollo y planteamiento de un diseño de una aplicación para este Trabajo de Fin de Grado (TFG) se pretenden alcanzar y lograr diversos objetivos, de manera general y específica, además de plantearme ciertos objetivos personales, los cuales pretendo alcanzar con la elaboración de este proyecto.

En cuanto a los objetivos personales que me planteo lograr, destaco los siguientes:

- Investigar el campo de la Inteligencia Artificial (IA).
- Mejorar un sistema genérico implantado en la sociedad, como es la seguridad vial.
- Mejorar habilidades de programación, especializándome en temas concretos.
- Crear un sistema que sea funcional propio, que sea escalable.
- Descubrir e investigar nuevos programas y tecnologías.

1.3.1. Objetivos generales

Como ya hemos mencionado, para realizar todos los trabajos, es bueno plantearse ciertos objetivos que definan el alcance del proyecto, es por lo que, el objetivo general que me planteo es diseñar una aplicación de gestión de tráfico eficiente.

A través de este objetivo general pretendemos desarrollar, progresar y fomentar un sistema de gestión de tráfico eficiente, fluido y automatizado, empleando diversas tecnologías, entre las que destacamos la visión por ordenador y *Deep Learning* (DL).

1.3.2. Objetivos específicos

Una vez marcado el objetivo general del trabajo, es bueno plantearse otro tipo de objetivos que también se pretenden lograr de manera transversal, destacando los siguientes:

- **OE-01.** Optimizar la circulación en situación de aglomeración de vehículos.

Con este objetivo específico se pretende mejorar la fluidez del tráfico en tiempo real, reduciendo y optimizando la movilización y tiempos de espera, para ello será necesario el empleo de tecnologías innovadoras en las diferentes infraestructuras que existen en la conducción.

- **OE-02.** Desarrollar la detección de situaciones de emergencia.

En cuanto a este segundo objetivo específico, desarrollado a través de este proyecto, vamos a poder cambiar, mejorar y alternar la circulación dependiendo de las situaciones de emergencia que surjan en el entorno.

2. ESTUDIO DEL MERCADO

En primer lugar, para entender el presente trabajo y la finalidad con la que se desarrolla, es imprescindible conocer, estudiar y entender los principales aspectos, conceptos e ideas que se van a abordar a lo largo de todo el proyecto. Es por lo que se va a realizar un recorrido a través de diferentes terminologías necesarias para la mejor comprensión de la aplicación diseñada y su funcionalidad, así como las herramientas empleadas en su desarrollo.

Además, se pretenderá unir conceptos de este proyecto, con aplicaciones, webs u otros recursos que presenten similitud, semejanza o conexión entre el tema principal y el que abordamos a lo largo del trabajo y del proyecto desarrollado, la descongestión del tráfico en los cruces.

2.1. Conceptos relevantes del dominio de aplicación.

- **Interfaz de Programación de Aplicaciones (API).**

Se trata del conjunto de mecanismos que permiten a 2 partes de software comunicarse entre sí mediante el uso de un conjunto de definiciones y protocolos establecidos. Su arquitectura suele ser cliente y servidor, el cliente es el que envía las peticiones, y el servidor el que las recibe, procesa y devuelve la información requerida (Amazon Web Service [AWS], s. f.).

- **Create, Read, Update and Delete (CRUD)**

Hace referencia al nombre de las operaciones con las que el sistema interactúa con los datos, la palabra en si misma es un acrónimo, y se corresponde con las acciones: crear, leer, actualizar y eliminar (León et al., 2011).

- **Deep Learning (DL)**

Se trata de un subcampo del Machine Learning, el cual utiliza redes neuronales artificiales con múltiples capas, su objetivo es ser óptimo en la toma de decisiones complejas, tarea que consigue procesando datos y analizándolos, esto es posible ya que está diseñado para que simule el cerebro humano (Internatioal Business Machines [IBM], 2023).

- **Inteligencia Artificial (IA)**

La Inteligencia Artificial (IA) es una tecnología que puede operar de forma autónoma o en conjunto con otros sistemas, diseñada para resolver problemas que normalmente requerirían intervención humana. Esta tecnología se fundamenta en el aprendizaje automático y el aprendizaje profundo, permitiendo a las máquinas aprender y adaptarse a través de complejas redes neuronales (IBM, 2021).

- **Machine Learning (ML)**

El Machine Learning (ML) es un subcampo de las tecnologías computacionales, que forma parte de la Inteligencia Artificial (IA). Su función principal es aprender y mejorar a partir de un conjunto de datos proporcionados, tarea que ha sido posible gracias al desarrollo de métodos específicos que consiguen que logre dicha tarea mediante algoritmos de predicción y toma de decisiones, con la peculiaridad de carecer de intervención humana directa (González, 2018).

- **Traffic Control Interface (TraCI)**

Se trata de una Interfaz de Programación de Aplicaciones (API) que nos permite comunicarnos con el simulador Simulation of Urban MObility (SUMO). Gracias a ella podemos recuperar y modificar valores de la simulación en tiempo real, mientras esta se encuentra en funcionamiento, de esta forma se ajusta el comportamiento de la simulación a las necesidades requeridas (*TraCI - Documentación SUMO*, 2024).

- **Simulation of Urban MObility (SUMO)**

Se trata de un simulador de circulación de código abierto diseñado para crear, modificar y visualizar redes de flujo de tráfico. Incluye un conjunto de herramientas que facilitan la creación de escenarios personalizados, y dispone de una Interfaz de Programación de Aplicaciones (API) que facilita el control, en tiempo real, sobre los elementos de esta (*SUMO Documentation*, 2024).

2.2. Relación con proyectos con la misma funcionalidad

Como ya hemos abordado anteriormente, a continuación, trataremos de reflejar diferentes tecnologías que traten de resolver uno de los problemas más cotidianos que tiene población a la hora de circular en carretera. Para ello expondremos distintos recursos que hay en el mercado junto con una pequeña descripción de su principal finalidad, reflejando así la similitud de estas aplicaciones con la desarrollada para este trabajo.

- **Surtrac (Scalable Urban Traffic Control)**

Es un sistema de control del tráfico adaptativo que hace uso de Inteligencia Artificial (IA), con la finalidad de optimizar el flujo de la circulación en vía urbana. Su sistema, en tiempo real, adapta los tiempos del tráfico para conseguir un correcto tránsito en redes complejas.

Su alcance no se limita a mantener bajo control un solo tipo de vehículo, si no que, es capaz de tener en cuenta diversos elementos que influyen en el correcto funcionamiento de la fluidez de la vía, como son: los peatones, vehículos, ciclistas y transporte público (*Surtrac (Scalable URban TRAffic Control)*, s. f.).

Uno de sus mayores puntos fuertes, es que, dispone de un sistema adaptativo que ajusta las incertidumbres del volumen de flujo dentro de la vía, por lo que consigue mantener un ciclo que no tiene cuello de botella.

El objetivo que ataca este proyecto no es solo el hecho de conseguir una circulación más fluida, segura y responsable, también aborda otras problemáticas secundarias, aunque bastante importantes, como son: el desgaste en la superficie de la vía, el desgaste de neumáticos y reducción en la emisión de los vehículos, que implícitamente, lleva integrada la reducción del consumo de este.

En conclusión, este proyecto ataca de forma directa una mejora en el sistema de la circulación eficiente, abarcando problemas secundarios, que de manera individual no son gran amenaza, pero en el conjunto de la población que

hace uso de la vía genera un desgaste importante (*RapidFlow Surtrac Is Now Miovision Surtrac*, s. f.).

- **InSync**

El proyecto que propone InSync es un sistema de señalización automático y adaptativo. Su propuesta es algo distinta al resto, ya que disponen de un *hardware* especializado, que se adapta a los sistemas actualmente vigentes, lo cual desemboca, no en una modificación del sistema actual, si no en una integración con lo ya existente.

Su robot procesa datos de la circulación del tráfico en tiempo real, de la zona donde ha sido instalado, y se vincula con el resto de las instalaciones para poder compartir los datos de las vías, gracias a su precisa visión y procesamiento continuo de datos, adapta el flujo de manera personalizada sincronizando las señales correspondientes para la mejora de este.

Su sistema no se limita a trabajar con los datos basados en la recolección de los últimos minutos, si no que, detecta, busca y estudia patrones de tráfico a medida que van surgiendo en la carretera.

En resumen, nos encontramos frente a un sistema revolucionario, que aboga por la integración y mejora del *hardware* existente, en lugar del reemplazo total del mismo, por otro lado, su procesamiento de patrones hace que no sea un sistema limitante, ya que no busca solucionar un problema de circulación de ahora mismo, todo lo contrario, busca crear patrones de circulación para resolverlos de manera genérica (*In|Sync*, s. f.).

- **Intelligent Traffic System en Milton Keynes**

Cuando hablamos de la optimización del tráfico en la vía urbana, de forma general, centramos nuestro pensamiento en la mejora de la señalización de las carreteras, pero existe otro punto de vista fundamental, como la mejora e implementación de la Inteligencia Artificial dentro de los propios vehículos, y es que, este proyecto abarca las dos alternativas.

Milton Keynes es una ciudad de Reino Unido que ha sido seleccionada para testear un transbordador autónomo. Este vehículo ha sido modificado e incorpora diversos sensores y cámaras que le aportan una visión de navegación de 360 grados, lo que permite mantener bajo control todo el perímetro que habita a su alrededor. Lo que se pretende conseguir con esta simulación viviente de transporte público, es que logre realizar un recorrido completo y autónomo por la ciudad, recorriendo las distintas paradas que son visitadas diariamente (*Milton Keynes City Council*, 2023).

De otro lado, se dispone de una implementación de señales de tráfico apoyadas por Inteligencia Artificial (IA), que trata de mantener bajo control la correcta circulación de peatones, ciclistas, motoristas y demás vehículos.

Como ha quedado reflejado, es un proyecto que ha creado un entorno de seguridad múltiple, manteniendo la temática principal de este Trabajo de Fin de Grado (TFG), pero que añade como extra, al entorno de la seguridad, la automatización de vehículos en el transporte público (*Milton Keynes City Council*, s. f.).

- **GreenWave**

Nos encontramos frente un proyecto cuyo diseño permite optimizar la fluidez del tráfico en las carreteras. Se centra en la posibilidad de poder abordar diversos cruces de forma sucesiva y continua, donde no solo se busca descongestionar las calles, si no, también poder aliviar el estrés ocasionado por este entorno y sus causas, reduciendo de manera considerable ruidos, contaminación y retrasos.

La manera en la que se pretende disminuir este mecanismo de parada y arranque, que nos vemos obligados a hacer con nuestros vehículos en situaciones de mala circulación, es gracias al término que ellos han denominado “ondas verdes”, y que se consigue con la posibilidad de cruzar las intersecciones libremente, ya que, varios semáforos consecutivos se encuentran coordinados.

Para que este flujo sea controlable, han tenido en cuenta a todos aquellos individuos que pueden participar dentro de la vía como: peatones, automóviles, bicicletas, tranvías, etc. Y deben de asegurarse de dos cosas principales, en

primer lugar, generar agrupaciones que abarquen a todos los individuos mencionados anteriormente, y en segundo lugar, crear flujos que se coordinen con los distintos grupos generados.

Es un procedimiento complejo de implantar, ya que una gran cantidad de factores entra en juego, pero si se consigue una regulación estable es un sistema eficiente. Nos encontramos frente a un procedimiento que no aborda la problemática presentada con la misma tecnología que los anteriores, pero que sí puede llegar a conseguir resultados parecidos si se plantea y ejecuta de forma correcta (SWARCO, s. f.).

- **Audi Traffic Light Information System**

Este sistema de información de semáforos lanzado por Audi es un servicio desarrollado en Las Vegas, que conecta los coches de su marca con la infraestructura de la ciudad, acción que permite regular la conducción del vehículo en base a su entorno.

En este caso particular, nos encontramos con un sistema que está montado de forma inversa, respecto a los casos desarrollados anteriormente, siendo el vehículo el que recibe la información del entorno, y te ayuda a regular tu conducción para que consiga ser más eficiente, segura y controlada.

Para poder conseguir dichos resultados, ha sido necesario cooperar con distintas ciudades de América del Norte, y actualizar las infraestructuras de tecnología. Es ahí cuando el coche puede recibir la información de todo lo que le rodea. También entra el factor humano en este desarrollo, ya que gracias a esta información el usuario es quien tiene el control de la circulación, y decidirá cuales son las mejores rutas y decisiones que debe tomar durante la práctica de la conducción.

Como conclusión podemos determinar que este método, es cuanto menos curioso, pero es una forma muy interesante, a la par que distinta, de adaptarnos al entorno que nos rodea. Es una gran opción tener controlada la información de lo que va a suceder de antemano, lo que proporciona mayor tranquilidad a la

persona que va a los mandos del vehículo (*Audi Networks with Traffic Lights in the USA*, 2016).

- **CITRAM**

El proyecto CITRAM se está desarrollando a nivel nacional en la Comunidad de Madrid, y está centrado en la red de sistemas de transporte público, teniendo como objetivo supervisar, regular y mejorar la circulación de este. Para ello, es necesario la unión de diversas empresas que aportan datos para mejorar la coordinación y tomas de decisiones dentro del proyecto (ESMARTCITY, 2016).

El recorrido a lo largo de todos los proyectos revisados en los apartados anteriores deja constancia de que no existe una forma correcta y óptima de implementar la Inteligencia Artificial (IA) dentro de la seguridad vial. Pero sí que marca una gran mejora en el rendimiento de la circulación en contextos urbanos, en todos los casos podemos observar unas mejoras genéricas, que son parte de los objetivos buscados por estos proyectos, como pueden ser: optimización de tiempos, mayor seguridad y reducción de contaminación y consumo.

2.3. Estudio de viabilidad

Para poder realizar el desarrollo de este proyecto, es primordial conocer diferentes enfoques que nos ayuden a seleccionar, comprender y conocer cuál es la idea más acertada, el alcance, alternativas y solución de los diferentes proyectos destinados a solventar esta problemática que llevamos planteando desde el inicio del trabajo, todo ello será enfocado y aplicado a la situación actual en la que nos encontramos.

2.3.1. Alcance del proyecto

Este proyecto se basa en la creación de una aplicación que captura los datos de los cruces de tráfico, que tienen semáforos, en tiempo real. Para poder simular la circulación del tráfico se ha generado un mapa y diseñado un escenario haciendo uso de un simulador de tráfico, con el que se representa un cruce real con flujo de vehículos en transición.

De forma paralela encontramos, en ejecución, un modelo de Inteligencia Artificial (IA) personalizado. Este ha sido entrenado expresamente para la detección y reconocimiento de vehículos, y se conecta de manera remota a la cámara del dispositivo móvil, simulando una cámara de vigilancia de tráfico de la vida real.

El modelo de IA se ejecuta de forma paralela a la simulación de tráfico y recoge los datos captados por la misma, a través de unas regiones contadoras de vehículos, de tal manera que se pueda identificar con exactitud cuántos vehículos disponemos en cada una de las carreteras del cruce.

Una vez los datos han sido captados por el modelo de IA, será necesario emplear un algoritmo que estará diseñado y basado en umbrales, a este se le asignará un tiempo de espera específico, basándose en la cantidad de vehículos que podemos encontrar en el cruce, de tal forma, que gracias a la Interfaz de Programación de Aplicaciones (API), la cual es específica del simulador de tráfico, nos permitirá cambiar los elementos y parámetros de la vía, mientras el programa está en ejecución.

Finalmente, con este enfoque, estableceremos un sistema integral que abarca la detección, el procesamiento, la reorganización y la implementación de medidas para el tráfico en tiempo real.

2.3.2. Estudio de la situación actual

A pesar de tener claros ejemplos alrededor del mundo, que puedan ayudar a resolver el problema planteado a lo largo del trabajo, vemos que la ejecución, en cada uno de ellos, es muy diferente. Centrándonos a nivel nacional podemos observar que carecemos de proyectos que tengan en el punto de mira y como principal finalidad llegar a solventar esta problemática. Como ya hemos mencionado anteriormente, el único proyecto destinado a ayudar a gestionar la circulación es CITRAM, dicho proyecto solamente es aplicado e implantado en la comunidad de Madrid, y tampoco está destinado a la regulación general de la circulación, si no únicamente a gestionar y supervisar el sistema de transporte público de dicha comunidad.

En definitiva, podríamos decir que gracias a la aplicación presentada lograremos conseguir un sistema que ayude a mejorar la fluidez del tráfico, llevando a otro nivel más amplio, genérico y autónomo la supervisión del tráfico en cualquier tipo de vía, siempre que tenga semáforo.

2.3.3. Estudio y valoración de las alternativas de solución

Como ya hemos comprobado existen diversos proyectos que abarcan la temática principal de nuestro proyecto. Entre estos proyectos podemos destacar CITRAM a nivel nacional, mencionado anteriormente, y otros proyectos que se pueden aplicar a mundial pero que no son puramente diseñados en España.

Dentro de estos últimos podemos encontrar distintas iniciativas como *Google Research*, con su iniciativa *Green Light*, la cual se encarga de reducir y optimizar las paradas de los vehículos en los semáforos, así como la emisión de gases en las zonas urbanas. Para poder llevar estas acciones a cabo *Google Research* utiliza datos de *Google Maps* para modelar, ajustar y organizar diversos modelos de tráfico.

Para finalizar, debemos de destacar que este proyecto todavía no ha sido implantado en España, pero si podemos encontrar 12 ciudades alrededor del mundo que ya cuentan con dicho sistema, entre las que destacamos: Abu Dhabi, Budapest, Manchester y Seattle.

En resumen, no existe ninguna solución o alternativa viable para la resolución de nuestro principal problema con el tráfico, es por lo que esta búsqueda intensiva de diferentes alternativas fue lo que me llevó a plantear el desarrollo de este proyecto, y así poder crear y enfocar un proyecto eficiente, factible y aplicable a nivel nacional, sin la necesidad de depender de productos de grandes marcas que solo llegan a países limitados.

2.3.4. Selección de la solución

Tras la revisión, estudio y formación respecto a los proyectos anteriores que tienen similitud con el objetivo que se pretende alcanzar en este Trabajo de Fin de Grado (TFG), se ha llegado a la conclusión, de que realmente, ninguno de ellos subsana el núcleo de lo que se pretende conseguir, de alguna manera

indirecta se consigue solucionar la problemática presentada en este proyecto, pero realmente no es lo que pretenden conseguir como principal objetivo, y por ende no se consigue en su totalidad la subsanación del problema.

Los proyectos que realmente son capaces de cumplir los requerimientos que pretendemos conseguir, y de hecho algunos de ellos ya se encuentran implantados en sus respectivos países, en ningún caso se encuentran implantados en España, ni si quiera por medio adaptativo.

Por las razones que se han mencionado en los párrafos previos, se plantea una alternativa que da solución directa, concreta y personalizada mediante el desarrollo de una aplicación que hace una detección en tiempo real de la afluencia del tráfico y siendo capaz de regularlo. Aunque dicho proyecto puede presentar alguna que otra limitación, puede tener algunas funcionalidades que no han sido antes abordadas por ninguna otra aplicación.

3. METODOLOGÍAS USADAS

La elección de una metodología que se adapte a las necesidades del proyecto es fundamental para una correcta elaboración de este, antes de seleccionar una metodología en concreto debemos hacer un pequeño recorrido para estudiar qué tipo debemos escoger.

La metodología elegida marcará el camino que se va a seguir en desarrollo del proyecto, y aunque se distingan diversos tipos de metodologías, hay unos requisitos que todo proyecto debe cumplir.

En todo momento es fundamental tener una visión del producto deseada, y si es un producto destinado a un cliente, es muy importante generar un vínculo con él. Además, en función de la tarea que vayamos a realizar tendremos que establecer su ciclo de vida, es decir, cuál será el transcurso desde que surge la idea hasta que se lleva a cabo de forma completa.

Por ello tomaremos como requisitos básicos todas aquellas funcionalidades y decisiones que se desean implementar en el proyecto, por ello es recomendable escribir un documento donde queden reflejadas todas las necesidades. Tras esto, se deberá establecer un plan que ejecute todas las acciones necesarias que den forma a estos requerimientos y visualizar su integración dentro de la aplicación. Siendo fundamental e importante definir ciertos límites, establecer normas y crear planes que garanticen que el proyecto avanza de forma adecuada y mantiene la calidad deseada.

Para terminar, algo que no debemos dejar pasar son los cambios, ya que no siempre será posible ejecutar aquellas tareas planteadas en la teoría, por lo que debemos tener en el punto de mira una gestión que permita añadir cambios sin necesidad de romper toda la aplicación, ajustándose a los tiempos y fechas límite establecidos previamente (Tinoco et al., 2014).

3.1. Metodología Tradicional

La metodología tradicional surge por la necesidad de crear un proyecto de calidad basándonos en unos estándares definidos. Tiene una visión a futuro donde se determina, de forma secuencial, cómo será el proceso de desarrollo

del proyecto, y lo hace captando requisitos necesarios para saciar los requerimientos del cliente, una vez definidos y especificados, con lo que se desarrolla el plan general.

En esta metodología se definen las fases en las que se dividirá el proyecto, cómo se resolverá cada apartado, y se estudian desviaciones, cambios y seguridad. Como hemos comentado, los requisitos son el primer paso para definir el camino de nuestro proyecto, por lo que será primordial saber si dichos objetivos son alcanzables, y si no lo son buscar alternativas y soluciones acorde al nivel deseado.

En cuanto a la documentación del proyecto debe de ser un proceso que debemos realizar de manera exhaustiva, ya que será el punto de referencia que utilizaremos para comprender cómo funciona todo el desarrollo que hemos comentado hasta ahora. En ella quedará establecido todo lo relacionado al proyecto, desde requisitos, modos de implantación e incluso cómo se ha desarrollado el proyecto (Cadavid et al., 2013).

3.1.1. Metodología en cascada

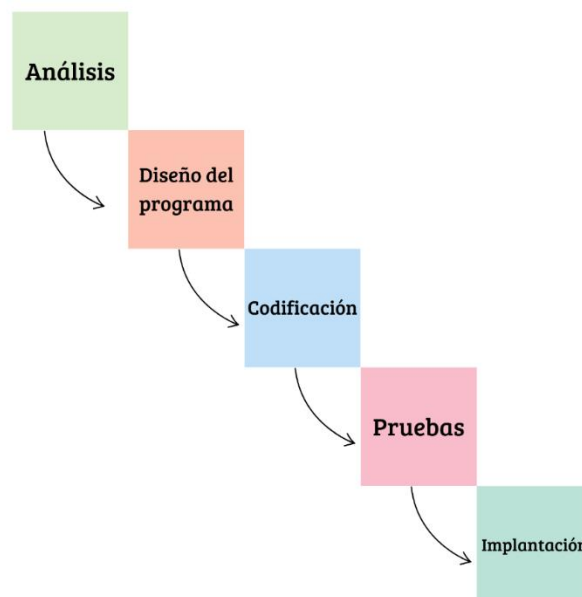
La metodología en cascada pretende resolver cada fase de una forma secuencial, es decir, se establecerán unas etapas que determinen la forma de resolver el proyecto propuesto.

Estas etapas se ejecutarán una detrás de otra, y nunca debemos iniciar una antes de que otra haya finalizado, además debemos designar un equipo especializado cuyos conocimientos estén acordes a los requerimientos de las tareas propuestas, llegando a crear subgrupos para fases o partes de las que se componga una tarea (Trigas, 2012).

Cada una de la persona asignada en una parte del trabajo tendrá asignado un rol específico, llegando a encontrar alguna persona con más de uno, lo que dependerá de factores como el tamaño de la empresa o las habilidades del individuo. A continuación, tal y como señala Trigas (2012), se mencionarán y definirán los diferentes roles que se pueden encontrar en esta metodología:

- **Gestor de proyectos:** es el encargado de controlar que el proyecto cumple los requisitos y planificaciones deseadas, desde el principio hasta el fin de su desarrollo.
- **Arquitecto de software:** son los encargados de recolectar, definir y diseñar los requisitos acordados.
- **Desarrolladores:** las personas con esta función se encargarán de llevar a cabo las instrucciones definidas por el arquitecto de *software*, asegurando el testeo y mantenimiento del desarrollo.
- **Probadores:** se encargarán de realizar las pruebas necesarias para asegurarse de que el programa cumple con los requisitos solicitados por el cliente.
- **Consejeros:** es una persona que no tiene una especialidad técnica, con la función de ayudar a comprender la problemática del proyecto, y así poder guiar y orientar hacia una solución óptima.

Figura 1. Esquema del funcionamiento de la metodología en cascada



Nota: Esquema visual para poder tener una imagen más clara de las partes que conforman esta metodología. Elaboración propia basándonos en la imagen del documento *Metodología Scrum* (p.16), por Manuel Trigas Gallego (2012).

3.1.2. Metodología RUP

La *Rational Unified Process* (RUP) es una metodología flexible, permitiendo, a las organizaciones y desarrolladores, ajustar los procesos de trabajo de acuerdo con las necesidades y requisitos específicos del proyecto. Este modelo se inspira en otras tecnologías, como el modelo en espiral, Trigas (2012, p.17) señala que “se basa en tres módulos principales que responden a las siguientes preguntas: quién lleva a cabo el proceso, qué productos de trabajo se van a generar, qué documentos y modelos se van a producir, y cómo se van a realizar las tareas”.

Una vez introducido este modelo, vamos a exponer cómo es su ciclo de vida, el cual se divide en cuatro fases iteradas. La primera fase es el inicio del proyecto, en esta se determinará el objetivo final, recogiendo los requisitos que marcaran el camino del resto del equipo.

Una vez realizado el estudio inicial, se procederá a la elaboración del plan. Para ello, debemos juntar la información recolectada para crear nuestro camino de forma teórica, considerando diversos factores que pueden afectar al desarrollo y generar alternativas para que una vez se implemente pueda probarse.

Trazado nuestro camino teórico, continuamos con la fase de construcción, en la cual, debemos llevar a la práctica todo lo acordado en el apartado anterior, para ello desarrollaremos el proyecto creando versiones utilizables que se ajusten a las necesidades que se pretenden cubrir.

Por último, se llega a la etapa de la transición, en ella debemos asegurar que toda la funcionalidad y estética ha sido alcanzada con éxito para poder obtener un *feedback por parte* del cliente (Trigas, 2012).

3.2. Metodología ágil

Las metodologías ágiles surgen en base a las metodologías tradicionales, al contrario de estas, que siguen un proceso iterado para alcanzar el desarrollo del proyecto, las metodologías ágiles son flexibles lo que permite

modificaciones, como resultado de esto vemos que hacen que sea posible diferentes adaptaciones a las necesidades de cada equipo de trabajo.

La metodología de trabajo que sigue es la división del proyecto en subproyectos, asignando a cada grupo del equipo un fragmento correspondiente, esta división se hace en base a unos requisitos y tipologías lo que permite la división en proyectos independientes.

Se caracterizan por tener plazos de trabajo cortos, normalmente de dos a seis semanas, en las cuales, una parte primordial es la comunicación constante durante el proceso de desarrollo del proyecto con el cliente (Cadavid et al., 2013).

3.3. Metodología elegida

Una vez realizado el estudio por las definiciones, características y aplicaciones de los distintos tipos de metodologías que más se han ajustado a las necesidades de mi proyecto, voy a mencionar y explicar la seleccionada para el desarrollo de este Trabajo de Fin de Grado (TFG).

La metodología ágil SCRUM propone una forma de trabajo flexible y autogestionada, además al estar diseñada para entornos donde puede haber incertidumbre, es ideal para desarrollar este proyecto, pues no puedo determinar con total confianza cuales son los pasos por seguir que determinaran el resultado óptimo que espero, por otro lado, su metodología se basa en ciclos cortos denominados “*Sprints*”, lo cual me ayudará a organizarme de forma constante y de una manera muy accesible a cambios.

Existen tres componentes generales que debemos tener en cuenta a la hora de organizar un proyecto con esta metodología.

En primer lugar, tenemos las reuniones. Al inicio del proyecto debemos plantear con el equipo ciertos límites y normas de alcance y desarrollo del proyecto, debemos planificar un *Backlog*, es decir, un documento que especifique todos los requisitos para nuestra aplicación, y que estén ordenados en base a un sistema de priorización, además esta reunión inicial nos servirá de base para organizar el Sprint cero. Debemos establecer una serie de reuniones diarias que servirán de orientación y evaluación del estado actual del proyecto.

Como último paso de este primer componente, debemos realizar una revisión del Sprint, lo que nos servirá para ver que tal estamos avanzando y mostrar resultados que sean concluyentes.

En segundo lugar, debemos determinar los roles, primero expondremos aquellos que están comprometidos con el proyecto, entre los que destacamos:

- **Product Owner:** toma las decisiones que afectan al proyecto y es encargado de organizar la prioridad de los requisitos.
- **ScrumMaster:** es la persona encargada de asegurar que la metodología se está siguiendo de forma correcta.
- **Equipo de desarrollo:** son los responsables de la implementación directa del proyecto.

En cuanto a los roles no comprometidos, nos referimos a todas esas personas que están interesadas en el proyecto, pero que no están envueltas de forma directa, como son:

- **Usuarios:** son aquellas personas a las que va dirigido el producto.
- **Stakeholders:** son todos los beneficiarios del proyecto
- **Managers:** son aquellos que solamente participan en la elección de objetivos y requisitos del proyecto.

Una vez resaltados y explicados los distintos tipos de roles que podemos encontrar dentro de esta metodología, en tercer lugar, vamos a ver los elementos fundamentales de esta tecnología, que son:

- **Product Backlog:** es la lista de requisitos del proyecto ordenada por prioridad.
- **Sprint Backlog:** lista de tareas específicas de un sprint, incluyendo tiempos, prioridades y responsabilidades.
- **Incremento:** es la parte de producto útil, es decir, que se ha completado de manera correcta, entregada al final de cada Sprint.

Una vez introducidos todos los elementos necesarios para conocer de qué se compone la metodología SCRUM, vamos a desarrollar las fases de esta. Aunque se trata de una metodología ágil y tiene un desarrollo flexible que admite cambios, existe un orden en la organización predeterminado, para garantizar el correcto desarrollo del proyecto.

Para poder usar esta metodología en un desarrollo, debemos comenzar con la preparación del proyecto, centrándonos en los objetivos que queremos conseguir, y midiéndolos por esfuerzo necesario para conseguirlos, así como por prioridad de desarrollo.

Una vez tenemos claro el diseño del esquema general, debemos comenzar con la planificación del *Sprint*, para ello, se suele utilizar una técnica llamada *Planning Poker*. Esta técnica se usa para que todos los órganos del grupo participen sin influencia externa. Tras ello se deberá llevar a cabo una serie de reuniones con el objetivo de desarrollar de forma eficiente el *Sprint*, estas reuniones son:

- **Reunión de planificación:** se establece la carga de trabajo del *sprint* a realizar.
- **Reunión diaria:** esta reunión sirve para poner al día los avances del proyecto.
- **Reunión de revisión del *Sprint*:** se utiliza para presentar el trabajo completado y obtener *feedback* del cliente.
- **Reunión de retrospectiva:** sirve para establecer el estado del funcionamiento general de los *sprints*, y de esta forma visualizar las mejoras a futuro.

Como podemos observar esta metodología ofrece mucha flexibilidad de trabajo, mantiene una constante retroalimentación, tanto con el cliente, como con el resto de los compañeros de proyecto, y es muy susceptible a cambios, imprevistos y reorganización de tareas, además, gracias al trabajo basado en *sprints*, el flujo de trabajo es una carga que regula su volumen de forma progresiva y marcada por objetivos. Por eso he determinado que es la que más

se adecúa para el desarrollo de este Trabajo de Fin de Grado (TFG) (Trigas, 2012).

4. TECNOLOGÍAS Y HERRAMIENTAS UTILIZADAS EN EL PROYECTO

Para lograr alcanzar el correcto desarrollo de la aplicación propuesta, ha sido necesario el uso de diferentes tecnologías, así como, distintas herramientas que han ayudado a conseguir el objetivo establecido. A continuación, haremos un recorrido por todas ellas, y examinaremos el porqué de su elección y en qué punto han sido utilizadas.

4.1. Tecnologías

A continuación, vamos a determinar y explicar, cuáles han sido las tecnologías seleccionadas para resolver la problemática propuesta, en cada una de ellas explicaremos su funcionamiento y el porqué de su elección.

4.1.1. YOLOv8

Dentro de este proyecto se ha decidido incorporar la Inteligencia Artificial (IA) para el reconocimiento en tiempo real de vehículos circulando por la vía, para ello, se ha hecho uso de la tecnología You Only Look Once version 8 (YOLOv8), se trata de un modelo capaz de realizar acciones de: clasificación, detección, segmentación, traqueo y posicionamiento de imágenes.

Este modelo ha sido de gran ayuda para la evolución general del proyecto, ya que, gracias a la posibilidad de crear versiones personalizadas, basadas en entrenamientos específicos, hemos podido generar un modelo propio que se ajusta a nuestra necesidad de detección de vehículos en las vías urbanas.

Para poder realizar este entrenamiento personalizado, y alcanzar así los objetivos de precisión de tan alto nivel, que son necesarios para asegurar la exactitud de la aplicación, ha sido necesario emplear un equipo con características de alta categoría, que permitiesen generar un entrenamiento cargante, junto con una configuración óptima del mismo, tanto por la parte de los

datos con los que lo hemos alimentado al modelo, así como la configuración que se ha establecido para conseguir unos resultados óptimos.

4.1.2. *TraCI*

Uno de los factores clave que teníamos como objetivo en el desarrollo de esta aplicación, era la detección, lectura y modificación de los datos en tiempo real. Por ello ha sido necesario conectar nuestro modelo personalizado de Inteligencia Artificial (IA), con un programa de simulación de circulación, para iniciar un caso real.

Para conseguir este resultado hemos utilizado la tecnología Application Programming Interface (API) del simulador SUMO, llamada Traffic Control Interface (TraCI). Gracias a ella conseguimos un control dinámico de la manipulación del tráfico entre el programa de control y el simulador.

Este control de la información, a través de esta interfaz, ha sido una pieza clave a la hora de modificar los parámetros necesarios dentro de nuestra ejecución, lo que nos ha permitido generar un vínculo que conecta ambas partes del programa sin necesidad de tiempos de espera, retrasos o errores.

4.1.3. *PyTorch*

Debido a la gran carga computacional, requerida para el entrenamiento personalizado de nuestro modelo de Inteligencia Artificial (IA), se ha requerido de la librería de PyTorch. Esta decisión se debe a varias razones clave que hacen de esta herramienta un imprescindible en el campo del Deep Learning (DL). Teniendo en cuenta su sencilla compatibilidad para ejecutar modelos con la GPU, ha aliviado de manera significativa la tarea del entrenamiento del modelo personalizado.

4.2. **Herramientas de desarrollo**

Como ya hemos mencionado, vamos a explorar cada una de las herramientas utilizadas en el desarrollo de la aplicación, de forma individual, tienen un papel clave que ha favorecido la tarea del estudio, conexión e implementación de las distintas tecnologías comentadas anteriormente.

4.2.1. *Visual Studio Code*

El entorno de programación que ha sido seleccionado para llevar a cabo el desarrollo de esta labor ha sido Visual Studio Code, ya que es una herramienta que está en constante evolución y actualización. Tiene disponible un *Marketplace* de *plugins*, lo que facilitan mucho la forma de interactuar con las distintas herramientas y lenguajes utilizados, además de herramientas secundarias que favorecen la labor de la programación.

4.2.2. *Python*

La decisión de escoger un lenguaje de programación, que permitiera trabajar de forma conjunta con el procesamiento de un modelo de Inteligencia Artificial (IA) y otros programas, era una decisión fundamental en el proyecto.

Los factores que han hecho que tome como decisión final utilizar Python, no se basan únicamente en ser un lenguaje popular con el que ya había trabajado. La decantación de este programa se centra en la gran cantidad de bibliotecas y *frameworks* con los que este lenguaje opera, permitiéndome desarrollar un entorno enlazado e integrado, no solo entre herramientas internas, también me han facilitado la tarea de comunicar el programa con elementos externos, gracias a su extensión dentro de la comunidad de desarrolladores y su simplicidad a la hora de programar.

4.2.3. *Netedit*

Para llevar a cabo la creación de un caso real, en el que pueda ser ejecutado el programa, ha sido necesario el uso de una recreación de un flujo de circulación haciendo uso de un simulador de tráfico. NetEdit es una herramienta gráfica que brinda los instrumentales necesarios para editar y diseñar redes de transporte.

Con esta herramienta se ha simplificado mucho la tarea de recrear los distintos escenarios a los que se ha enfrentado el modelo de Inteligencia Artificial (IA). No solo permite la implementación de elementos visuales, también es posible la configuración de los elementos que no vemos por pantalla, como las rutas, flujos y configuración de los componentes propios de una vía. El uso de

archivos XML facilita la creación de escenarios mediante código, mejorando la eficiencia y precisión en el diseño de entornos de simulación.

4.2.4. SUMO

En el apartado anterior hemos estudiado la herramienta utilizada para crear redes de transporte, estando relacionada de forma directa con el simulador sobre el que se va a reproducir. Los motivos por los que se ha decidido utilizar Simulation of Urban MObility, han sido tres.

En primer lugar, este simulador permite una fácil personalización de sus elementos, algo que nos ha aportado mucho valor ha sido la posibilidad de modificar el modelado de los coches, siendo este sustituido por imágenes más realistas que aportan realismo a la ejecución.

Y en segundo y tercer lugar, encontramos dos elementos ya mencionados y desarrollados anteriormente, que son la API de TraCI, permitiéndonos comunicarnos en tiempo real entre el programa base y la simulación, y la aplicación NetEdit, que nos ha permitido modificar el mapeado de la simulación siguiendo las necesidades requeridas por la aplicación.

4.2.5. Irium Webcam

Esta es una herramienta tan sencilla, como necesaria dentro del circuito, se trata de una aplicación que ha permitido conectar la cámara de mi teléfono móvil al ordenador, ya que este no dispone de cámara. Gracias a ella las imágenes captadas por el móvil, son transferidas de forma inmediata al ordenador para poder ser procesadas.

4.3. Equipo Hardware

Tras una revisión del software principal en el que se basa el desarrollo de la aplicación, recorreremos otra parte fundamental de este proyecto, que nos ha apoyado en la dura tarea del procesamiento de los datos.

4.3.1. Ordenador de sobremesa personalizado

Dentro del desarrollo de la aplicación hay tareas que requieren, no solo de un *hardware* especializado, sino también de una gran capacidad de procesamiento. En este caso se ha utilizado un ordenador de sobremesa montado por piezas, entre las cuales destacamos las siguientes:

- **Procesador:** Ryzen 7 3800xt.
- **Memoria RAM:** 16 GB DDR4.
- **GPU:** NVIDIA Geforce RTX 2060.
- **Almacenamiento interno:** 1 TB SSD.

4.3.2. Nothing Phone 1

Este dispositivo móvil ha sido utilizado para la función de cámara, es el encargado de grabar la simulación y enviar las imágenes en tiempo real al ordenador, a través del *software* mencionado anteriormente, para que puedan ser procesadas y utilizadas dentro del proyecto.

4.4. Software de ayuda

Los softwares de gestión de proyectos que ayudan a controlar las versiones del programa ofrecen tranquilidad y facilidad para manipular el proyecto. Son fundamentales para poder hacer un seguimiento de las distintas ramas donde se desarrollan etapas de pruebas, y nos ayudan a protegernos de la pérdida de los datos. A continuación, presentamos los utilizados en este trabajo.

4.4.1. Git

Este es un sistema de control de versiones distribuido, que nos ha permitido realizar un seguimiento, en cuanto a los cambios del programa se refiere. Especialmente ha sido usado para indagar en distintas variantes, durante el desarrollo de la aplicación, ya que, en muchas ocasiones existían caminos

distintos que llevaban a soluciones interesantes, y esta era la mejor forma de mantenerlos seguros y poder hacer pruebas.

4.4.2. GitHub

GitHub es un portal que permite crear repositorios donde alojar nuestro código, estos pueden ser públicos o privados, y pueden ser compartidos con el resto de nuestro equipo de trabajo. Hace uso de Git y ofrece un entorno web amigable con el que poder trabajar. En diversas ocasiones ha sido utilizado para búsqueda de información, ya que su interfaz permite estructurar los proyectos, de manera que otros usuarios puedan hacer uso de ellos.

En conclusión, estas han sido todas las herramientas, tecnologías y programas que, en su conjunto y colaboración, han dado vida a la aplicación, cada una de ellas tiene su papel fundamental, dentro del desarrollo, implementación y testeo del programa.

5. ESTIMACIÓN DE RECURSOS Y PLANIFICACIÓN

Tras la revisión alrededor de todos los conceptos, tecnologías y metodologías, que se han utilizado para el desarrollo de este proyecto, se va a exponer cuáles han sido las tareas ejecutadas que han concluido en un sistema completo, autónomo y escalable. Teniendo en cuenta la metodología ágil Scrum, se van a resolver todos los *Sprints* planteados para la correcta evolución de este Trabajo de Fin de Grado (TFG).

5.1. Sprit 0

Tal y como se ha explicado anteriormente, cuando se ha hablado de la metodología Scrum, este *Sprint* es una fase preoperatoria que define el proyecto antes del comienzo de los *Sprints* comunes. Con ello, se pretende establecer una base sólida sobre la que trabajar, disponiendo de un trabajo estructurado que nos permita reconocer de antemano las necesidades que van a surgir en nuestro desarrollo.

5.1.1. Planificación del Sprint

Para mantener una correcta organización a lo largo de todo el proyecto, se necesita partir de una base sólida, es por lo que vamos a definir una serie de tareas de análisis que se encargaran de recoger todas las necesidades que nuestro proyecto requiere:

1. Especificar requisitos.
2. Planificar el *Product Backlog*.
3. Estimar las horas de trabajo.
4. Estimar los costes del proyecto

Una vez definidas las tareas que se deben realizar, previas al desarrollo de los *Sprints*, expondremos como se ha resuelto cada una de ellas y cuáles han sido las conclusiones que nos han proporcionado.

5.2. Especificación de requisitos

A la hora de trabajar con la metodología Scrum, un paso fundamental de esta, es el *Product Backlog*, el cual, define todas aquellas tareas a realizar durante la ejecución del proyecto. Para ayudarnos a definir dichas tareas hemos realizado una captación de requisitos que definen la estructura del trabajo.

El objetivo que se pretende conseguir con este Trabajo de Fin de Grado (TFG), es la creación de una aplicación, programada en Python y que regule la circulación del tráfico en los cruces de los semáforos. Para conseguir este resultado, se ha realizado una división en dos requisitos generales, subdivididos a su vez, en requisitos más pequeños.

- **R-01.** Desarrollar un modelo personalizado de Inteligencia Artificial (IA) capaz de detectar vehículos en circulación.

- **R-01.1.** Selección y etiquetado del *dataset*.

Este punto se refiere a la recopilación de los datos que nutrirá a nuestro sistema de detección de vehículos. Consiste en la recolección de imágenes que son clave para la ejecución del programa, y la adaptación de estas para que puedan ser evaluadas en el entrenamiento. Hay que destacar que al tratarse de un programa que necesita una alta precisión en su ejecución a tiempo real, el volumen y la calidad de los datos presentados debe ser alto.

- **R-01.2.** Entrenamiento personalizado del modelo.

Una vez que disponemos de un conjunto de datos compatible con el modelo con el que vamos a crear nuestra versión personalizada, hay que realizar un estudio que nos asegure una ejecución óptima. Para ello debemos asegurar que los parámetros de la configuración del entrenamiento se adaptan para conseguir el resultado esperado, y una vez esto esté listo, procesarlo todo para que dé como resultado un modelo capaz de captar vehículos en circulación.

- **R-02.** Desarrollo de una aplicación que integre los sistemas externos.

- **R-02.1.** Diseño e implementación de la simulación.

Utilizando las tecnologías y herramientas de simulación mencionadas y desarrolladas anteriormente, crear una red de carreteras que permita ejecutar nuestro programa en un entorno lo más parecido a la vida real.

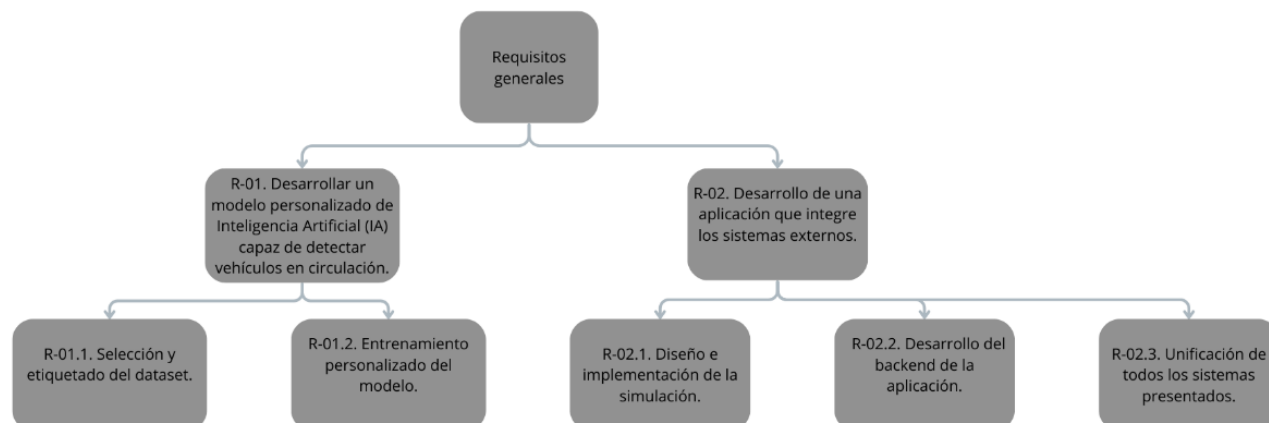
- **R-02.2.** Desarrollo del *backend* de la aplicación.

Mediante las técnicas de programación, y haciendo uso del conocimiento investigado, procesar un entorno que permita la ejecución del programa. Este será el cerebro de todo el programa y será el encargado de realizar los ajustes necesarios para que todo funcione correctamente.

- **R-02.3.** Unificación de todos los sistemas presentados.

Tras la ejecución de todas las piezas que componen este proyecto, diseñar un sistema que permita compartir y trabajar con la información ofrecida por todos los servicios, de tal manera que, tras la unificación todo funcione como un solo programa.

Figura 2. *Requisitos generales*



Nota: esquema de elaboración propia, de los requisitos generales que engloban el proyecto planteado.

5.3. Planificación del Product Backlog

Tras la recolección y definición de los requisitos que componen nuestra aplicación, es hora de dar forma al *Product Backlog*, en este, definiremos en forma de historias de usuario, todas aquellas tareas necesarias para cumplir con los requisitos. En primera instancia vamos a especificar dichas historias de una forma simple, que más adelante modificaremos para añadir prioridades y puntos estimados.

Tabla 1. *Historia de usuarios*

Historia de usuario: 001	Nombre: buscar imágenes
Criterios de aceptación: 1. Las imágenes deben cumplir los requisitos de similitud temática y calidad respecto al proyecto propuesto.	
Necesito: encontrar imágenes de vehículos circulando.	
Para: poder realizar un entrenamiento preciso.	
Historia de usuario: 002	Nombre: etiquetar imágenes
Criterios de aceptación: 1. El etiquetado del <i>dataset</i> debe ser compatible con la versión del modelo que creará nuestra versión personalizada.	

Necesito: etiquetar las imágenes.	
Para: que puedan ser reconocidas de forma precisa en el entrenamiento de datos.	
Historia de usuario: 003	Nombre: entrenar el modelo de datos
Criterios de aceptación: 1. El modelo resultado debe mantener un porcentaje de precisión elevado, en cuanto al reconocimiento de vehículos se refiere.	
Necesito: configurar y entrenar el modelo de detección.	
Para: poder detectar, en tiempo real, los vehículos que circulan por la vía.	
Historia de usuario: 004	Nombre: creación de la red de carreteras
Criterios de aceptación: 1. La red de la carretera debe mantener similitud cercana a la realidad.	
Necesito: generar una red de vías urbanas.	
Para: disponer de un escenario en el que ejecutar la simulación	
Historia de usuario: 005	Nombre: configuración del simulador
Criterios de aceptación: 1. El simulador debe poder utilizar imágenes de coches reales, ya que, el modelo de Inteligencia Artificial (IA) ha sido entrenado con esta tipología de datos.	
Necesito: configurar el simulador de forma óptima, cómoda y versátil.	
Para: que la representación de los vehículos en plena circulación sea lo más realista posible.	
Historia de usuario: 006	Nombre: desarrollo del programa base.

Criterios de aceptación: 1. El programa debe poder controlar todos los datos, ser capaz de gestionarlos, y ofrecer una solución óptima para el caso en el que se encuentre.	
Necesito: un programa que me permita controlar las distintas tecnologías.	
Para: disponer de un programa que haga de cerebro de la aplicación.	
Historia de usuario: 007	Nombre: integración de la API.
Criterios de aceptación: 1. El intercambio de información entre el simulador y el programa debe presentar tiempos de respuesta muy bajos.	
Necesito: intercambiar información entre el programa y el simulador.	
Para: aplicar los cambios generados por el programa, basados en la detección.	
Historia de usuario: 008	Nombre: integración de los sistemas.
Criterios de aceptación: 1. En ninguno de los casos se permite la no integración de una tecnología o herramienta que sea fundamental para abordar los objetivos mínimos deseados.	
Necesito: que todos los sistemas se integren como si fuesen uno solo.	
Para: que actúen en coordinación, de una manera rápida y eficaz.	

Nota: elaboración propia. Esta tabla recoge las distintas historias de usuarios.

Una vez hemos definido, de una forma sencilla, nuestras historias de usuario, necesitamos aplicar una técnica denominada *Planning Poker*, que nos va a ayudar a determinar la complejidad de la realización de dichas tareas, y, además, podremos establecer una estimación de prioridades para realizarlas. Antes de proceder a ejecutar dicha técnica, se va a explicar su funcionamiento.

Cuando utilizamos el *Planning Poker*, asignamos a cada uno de nuestros integrantes del grupo un mazo de cartas, estas cartas contienen una serie de valores, que son: 0, 1/2, 1, 2, 3, 5, 8, 13, 20, 40 y 100. Debemos de destacar que los valores numéricos no están puestos al azar, son valores redondeados de la

secuencia de *Fibonacci*, y suelen ser utilizados para la asignación de los puntos de historia, los cuales, indican el tiempo de realización de cada tarea.

La ejecución de la técnica es simple, se decide a qué tarea se le desea calcular el valor de esfuerzo, y cada uno de los integrantes, con su respectiva baraja y sin que el resto lo vea, decide asignar un valor de dificultad a la tarea planteada, este valor se basa en la opinión propia de cada uno. Esto se hace de esta forma para que cada persona pueda aportar los motivos por los que cree que a esa tarea le corresponde ese valor, este proceso se va repitiendo hasta que se llega a un consenso (Ortiz Tanori & García Mireles, s.f.).

Para poder disponer de un *Product Backlog* completo, debemos asignar también la prioridad de las tareas por cada *Sprint*. Para organizar esta parte vamos a hacer uso de una metodología llamada MoSCoW, que es un conjunto de normas que nos permiten seleccionar las prioridades de cada historia, estas van a ir ligadas a las propias letras que conforman la palabra:

- M: *must have*, son aquellas tareas imprescindibles en el proyecto.
- S: *should have*, hace referencia a las tareas de prioridad elevada.
- C: *could have*, son aquellas tareas que se deben considerar incluir.
- W: *won't have*, se indican las tareas que no van a ser realizadas.

Con todo lo visto anteriormente, ya estamos preparados para generar nuestro *Product Backlog* de forma completa, a continuación, encontraremos todas las tareas ordenadas por su Sprint correspondiente, incluyendo, gracias a los sistemas de prioridad y puntos de historia, sus correspondientes valores.

Tabla 2. *Product Backlog*

Product Backlog				
Historia de usuario	Sprint	Descripción de la tarea	Prioridad	Puntos de historia
6	1	Crear proyecto en	M	1/2

		Visual Studio Code		
6	1	Crear proyecto en NetEdit	M	1
6	1	Crear proyect en SUMO	M	1
1	1	Búsqueda de imágenes	M	8
6	2	Configuración del entorno, e instalación de librerías y dependencias	M	5
4	2	Crear red de carreteras	M	5
5	2	Conectar la red de carreteras con SUMO	M	2
6	2	Generar archivos xml que configuren la simulación a partir del código	S	3
2	2	Etiquetar las imágenes que	M	5

		se usarán en el entrenamiento		
3	3	Entrenamiento personalizado del modelo de Inteligencia Artificial (IA)	M	40
3	3	Detección de vehículos a través de vídeos	C	8
3	3	Sistema de detección de elementos basado en regiones	M	5
6	4	Sistema basado en umbrales para optimizar la circulación	M	8
8	4	Conexión entre el modelo entrenado y la cámara del móvil	M	5
7	4	Configurar la API de TraCI	M	8

8	5	Unión entre el programa base, la API y el modelo entrenado	M	20
6	5	Implantación de <i>multithreading</i> en la ejecución del programa	S	13

Nota: elaboración propia, creación de la organización del Product Backlog,

Llegados a este punto nos encontramos con un proyecto totalmente definido, gracias a la metodología ágil Scrum, hemos conseguido atribuir y repartir el tamaño de la carga, establecer las prioridades, y repartir el peso del trabajo en distintos *Sprints*.

5.4. Planificación temporal.

A continuación, se va a establecer un horario que sea capaz de cumplir con la programación determinada. Para poder mantener una metodología de trabajo semanal, que sea constante, a la vez que estable, se ha decidido repartir de forma igualitaria la carga de trabajo alrededor de la semana, dejando los fines de semana para descansar, siendo utilizado algún sábado si el proyecto lo requiere.

La idea que se persigue es realizar jornadas de trabajo de seis horas diarias de lunes a viernes, en función a las tareas que se vayan a resolver, o la carga de trabajo de estas, repartiremos dichas horas en dos formatos distintos. El primero sería realizar sesiones ininterrumpidas de seis horas, suponiendo que se trate de una tarea que requiere de una máxima atención y constancia de trabajo, y en segundo lugar trabajar en dos periodos de tres horas cada uno.

Tabla 3. *Horario de trabajo*

Lunes	Martes	Miércoles	Jueves	Viernes	Sábado	Domingo
6 horas	6 horas	6 horas	6 horas	6 horas	Opcional	Descanso

Nota: elaboración propia, establecimiento del horario semanal.

En principio se persigue mantener el segundo formato ya que facilita el reparto equitativo de tareas, y favorece el descanso mental. En cuanto a jornadas extra, se ha propuesto, en caso necesario, ampliar el horario de trabajo hasta el sábado, ya que nos vamos a encontrar con distintas problemáticas que seguramente requieran que aportemos más dedicación.

Si recuperamos los valores de los puntos de historia, establecidos anteriormente, y realizamos su sumatorio, obtenemos un total de 137.5 puntos. Con esta información podemos realizar una operación que nos permita determinar, de manera estimada, cuál será la velocidad del trabajo, así, será más sencilla la organización del trabajo a largo plazo. En el escenario base determinamos treinta horas semanales, seis diarias de lunes a viernes, con estos datos se puede calcular la velocidad del trabajo.

$$Velocidad\ de\ trabajo = \frac{Puntos\ de\ historia}{Horas\ totales}$$

Para poder aplicar esta fórmula hay que saber las horas planteadas para el desarrollo de este proyecto, considerando que realizamos seis horas diarias, cinco días a la semana, y suponiendo un tiempo de desarrollo de cuatro meses, obtenemos como resultado, 480 horas, por lo que ya podemos aplicar la fórmula vista anteriormente.

$$Velocidad\ de\ trabajo = \frac{137,5}{480} = 0,286\ ph$$

La velocidad de trabajo estimada, según los datos proporcionados, es de 0.286 puntos de historia por hora, esta cifra es meramente teórica, debido a que

Scrum es una metodología ágil, aunque este dato nos puede resultar útil para servir de guía durante todo el recorrido del desarrollo.

Con toda la información recolectada, es posible realizar una estructura del tiempo que se le va a dedicar a cada *Sprint*.

Tabla 4. *Estructura temporal del Sprint.*

Número del Sprint	Puntos de historia	Horas
1	10.5	36.71
2	18	62.93
3	53	185.31
4	21	73.42
5	33	115.38

Nota: elaboración propia, resumen de los puntos de historia y horas correspondientes de cada *Sprint*.

5.5. Estimación de costes

El coste total del proyecto supone un proceso en el que entran en juego varios factores clave para determinar con exactitud el valor de este. Podríamos utilizar datos y estadísticas reales que nos ayuden a acercarnos a la realidad, si nos basamos en salarios reales de programadores de *Python*, cuyo trabajo implica el uso de tecnologías con Inteligencia Artificial (IA), encontramos un valor medio de unos 39.024€ al año, de media. Teniendo en cuenta que este es un valor medio, y nuestra experiencia es baja podríamos tomar como referencia un valor de 25.000€ anuales.

Centrándonos en las personas que han dado valor al desarrollo de la aplicación, y han tenido un rol directo dentro de ella encontramos tres casos bien diferenciados. En primer lugar, encontramos la silueta del *Product Owner*, correspondiente con Jose Ángel Gallego García, en segundo lugar, tenemos el *Scrum Master* con Francisco Arcas Túnez, por último, el *Team* dentro del cual se involucra de nuevo a Jose Ángel Gallego García.

Tras el estudio del estado salarial actual, de los desarrolladores que abordan problemática de la misma temática que este proyecto, y realizado un recorrido por los integrantes que forman el grupo de trabajo, tenemos los elementos necesarios para poder realizar una estimación de costes.

Dicha estimación se obtiene a través de un sencillo cálculo, multiplicar las horas totales de tiempo de trabajo, por el coste de la hora de trabajo, este último, se puede averiguar dividiendo el salario anual, entre los días trabajados y sus correspondientes horas laborales.

$$\text{Salario por hora} = \frac{\left(\frac{\text{Salario anual}}{\text{Días laborales}} \right)}{\text{Horas de trabajo diarios}}$$

$$\text{Salario por hora} = \frac{\left(\frac{25000}{229} \right)}{8} = 13.6 \text{ €/h}$$

Llegados a este punto, tenemos todos los datos que permiten estimar un coste de proyecto, recordemos que el total de horas que finalice este trabajo son 480 horas, y gracias al cálculo del precio de la hora de trabajo, podemos determinar que tendría un coste de 6528 euros.

6. DESARROLLO DEL PROYECTO

Anteriormente, se ha realizado un recorrido argumentativo por todos los elementos que conforman este Trabajo de Fin de Grado (TFG). Se han desarrollado para entender, de una mejor forma, cuál es su funcionamiento, y el por qué se han decidido utilizar. También ha habido un estudio teórico del alcance, duración y coste del proyecto, lo que ha aportado valor organizativo, ofreciendo una visión de futuro estructurada. Es momento de centrarse en la aplicación de todos estos conocimientos, y evaluar, de una forma más concreta, cada paso de los Sprints propuestos, con ello quedará reflejada la aplicación práctica de los conocimientos desarrollados.

6.1. Sprint 1

Para poder llevar un mejor seguimiento de este Sprint, seguidamente, podemos visualizar cuáles son las tareas que se encuentran detalladas dentro del *Product Backlog*.

Tabla 5. Descripción Sprint 1

Historia de usuario	Sprint	Descripción de la tarea	Prioridad	Puntos de historia
6	1	Crear proyecto en Visual Studio Code	M	1/2
6	1	Crear proyecto en NetEdit	M	1
6	1	Crear proyect en SUMO	M	1
1	1	Búsqueda de imágenes	M	8

Nota: elaboración propia.

6.1.1. Planificación del Sprint

Este primer Sprint, tiene como objetivo general, la creación y configuración del proyecto, por ello mismo, veremos todos los pasos necesarios que han permitido establecer un entorno de trabajo óptimo.

Como primer paso debemos descargar el *software* de programación *Visual Studio Code*, y proceder con su instalación, mientras esta genera los archivos necesarios en el sistema, crearemos una carpeta que hará de directorio general de la aplicación.

Una vez ha finalizado la instalación, accederemos al *Marketplace* que ofrece este *software* para poder instalar una serie de *plugins* que nos facilitarán la tarea de la programación, también es recomendable, puesto que vamos a trabajar con el lenguaje de programación *Python*, generar un entorno virtual, el cual, nos permite tener un aislamiento en la instalación de paquetes y librerías, esto hará que si tenemos otros proyectos dentro de nuestro ordenador no nos influyan y corrompan el programa.

Una vez finalizada con la configuración de *Visual Studio Code*, debemos seguir con el programa *NetEdit*, este nos ayudará a crear las redes de carreteras de nuestra simulación de circulación de vehículos. Tendremos que descargar e instalar el *software*, y al igual que con *Visual Studio Code*, generaremos un directorio para almacenar los proyectos que desarrollemos en este *software*.

Esta red de carreteras debe ejecutarse en un simulador, por lo que es momento de descargar e instalar SUMO, en este caso, no es necesario dedicarle un repositorio, ya que lo comparte con *NetEdit*.

Por último, debemos iniciar una búsqueda de datos, en forma de imagen, que nos ayude con el entrenamiento de nuestro modelo de Inteligencia Artificial (IA). Para ello preestableceremos de una forma clara los requisitos y características que deben de tener las imágenes, y una vez que las tengamos seleccionadas debemos almacenarlas en un directorio para su posterior etiquetado.

6.1.2. Retrospectiva del Sprint

Puesto que este *Sprint* se ha centrado en la investigación, estudio y configuración de las herramientas base, no es posible destacar ningún problema, más allá de los contratiempos ocasionados por la falta de experiencia o desconocimiento. Cabe destacar que la búsqueda de imágenes, a pesar de no haber generado un contratiempo como tal, sí que ha requerido de una dedicación mayor al resto, ya que no todas las imágenes, relacionadas con la temática, eran válidas.

6.2. Sprint 2

Una vez disponemos de todas las herramientas instaladas, dedicaremos el tiempo de este *Sprint* a dar forma a nuestra aplicación, para ello visualizaremos las tareas por hacer de una forma más concreta en un fragmento del *Product Backlog*.

Tabla 6. Descripción Sprint 2

Historia de usuario	Sprint	Descripción de la tarea	Prioridad	Puntos de historia
6	2	Configuración del entorno, e instalación de librerías y dependencias	M	5
4	2	Crear red de carreteras	M	5
5	2	Conectar la red de carreteras con SUMO	M	2
6	2	Generar archivos xml	S	3

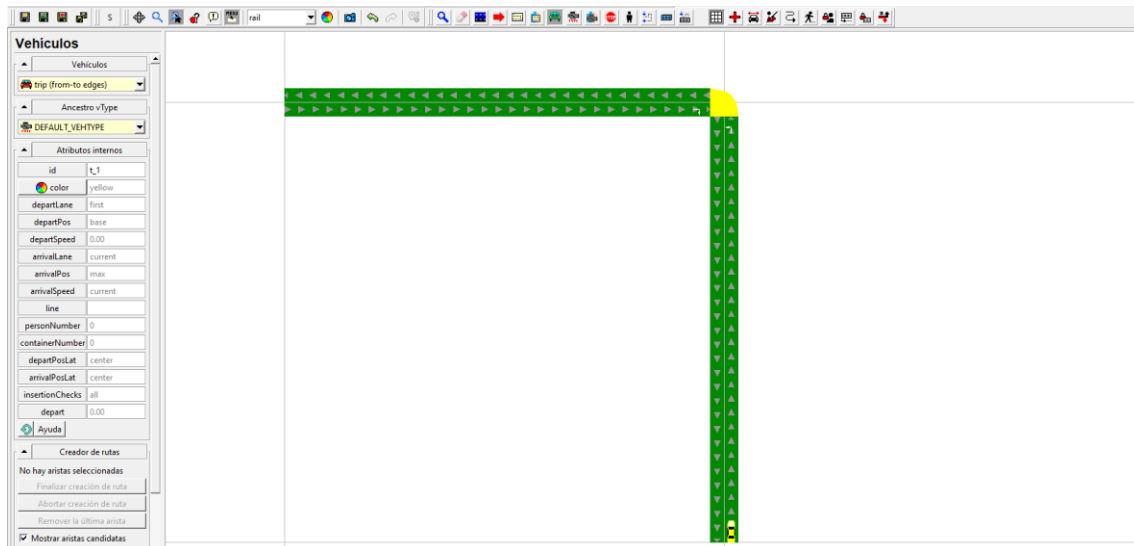
		que configuren la simulación a partir del código		
2	2	Etiquetar las imágenes que se usarán en el entrenamiento	M	5

Nota: elaboración propia.

6.2.1. Planificación del Sprint

Comenzamos este *Sprint* configurando todas las librerías y dependencias necesarias, lo que nos va a permitir desarrollar con normalidad nuestro trabajo. Destacamos dos librerías en concreto que son parte fundamental del proyecto, en primer lugar, la de *Ultralytics*, la cual nos permitirá, más adelante, generar nuestro modelo de Inteligencia Artificial (IA) personalizado, en segundo lugar, instalar *PyTorch*, este también nos ayudará con la creación de nuestro modelo personalizado, concretamente en la etapa del entrenamiento, puesto que nos permitirá hacer los cálculos haciendo uso de la GPU.

Una vez nuestro entorno está listo para trabajar, es hora de crear nuestra red de carreteras, para la cual, utilizaremos el *software NetEdit*, el objetivo es conseguir representar una vía realista, con una disposición de carril doble, en la cual dispongamos de cruces regulados por semáforos, esto último es una pieza indispensable del proyecto. Al igual que las carreteras, debemos incluir un flujo de circulación de vehículos, y establecer las rutas que nos permitirán generar los casos objetivo de estudio.

Figura 3. Red de carretera

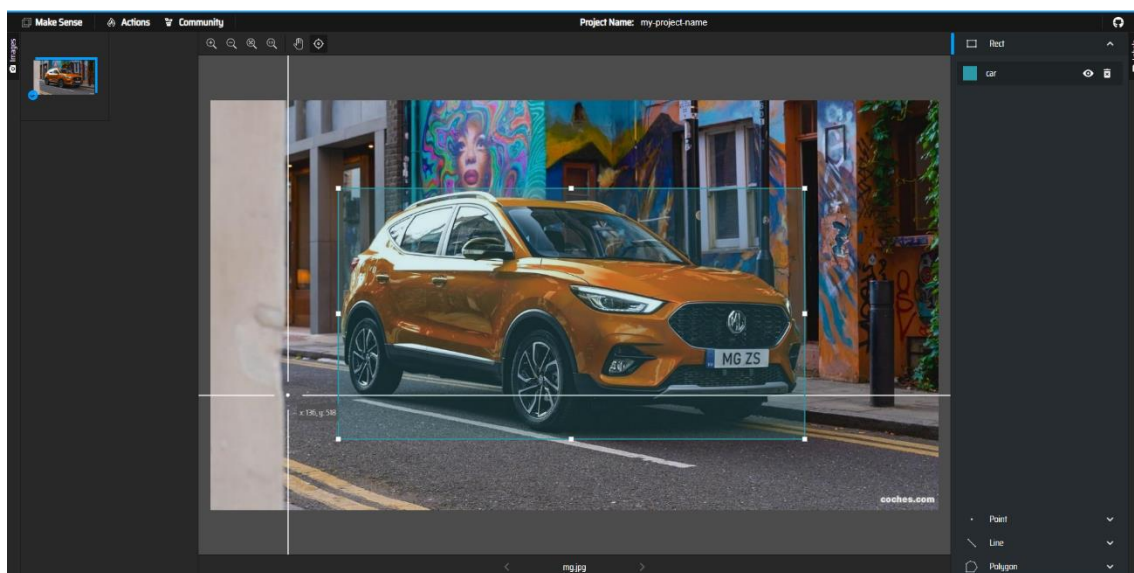
Nota: elaboración propia de un mapa con el uso del programa NetEdit.

Tras la creación y configuración del entorno de circulación de los vehículos, es necesario ejecutar pruebas dentro de SUMO que nos ayuden a conseguir el resultado deseado. Además, dentro de este, disponemos de otras opciones, orientadas más hacia el ámbito estético, que permite añadir más realismo a la simulación, entre otras opciones destacamos la posibilidad de modificar la velocidad de la simulación, y el modo “*real world*”, que añade realismo a los elementos estéticos.

Una vez se ha conseguido encontrar la disposición deseada, vamos a generar los archivos de configuración de *NetEdit* y SUMO. Con el empleo del código, el objetivo que se persigue con esta acción es la posibilidad de generar distintos casos de ejecución que nos permitan evaluar diversas situaciones y poner a prueba nuestro modelo personalizado, el hecho de hacerlo a través de código simplifica la tarea de tener que escribir archivos extremadamente largos de forma manual. Además, en este apartado también modificaremos ciertos parámetros de configuración de la simulación, como el cambio de modelo de coche por uno personalizado más realista, el fin de esta acción es que el modelo de Inteligencia Artificial (IA) se basa en datos de coches reales, y este simulador no utiliza modelos realistas.

Como último paso de este *Sprint* se realizará el etiquetado de las imágenes, este proceso es una tarea tediosa, ya que debe hacerse de manera manual e imagen por imagen. Para ello, se ha hecho uso de la plataforma web *Make Sense* que nos permite etiquetar imágenes con el formato que vamos a utilizar para el entrenamiento. También se ha hecho uso de otra plataforma web llamada *Roboflow*, la cual pone a disposición diversos *datasets* pre etiquetados. Esto no es solo beneficioso para ahorrar trabajo, si no para una mayor diversidad de tipología de datos.

Figura 4. *Etiquetado de un coche*



Nota: captura de pantalla del programa *MakeSense*, en el proceso de etiquetado de un coche, elaboración propia.

6.2.2. Retrospectiva del Sprint

En este *Sprint* ya podemos encontrar ciertas dificultades que han impedido el correcto avance, aunque hay que destacar que en ninguno de los casos se ha tenido que descartar la idea que se perseguía, ni ser sustituida por alternativas secundarias.

La tarea que más nos ha entorpecido ha sido la creación de distintos casos de configuración de *NetEdit* y SUMO a través del código, ya que, no disponíamos de la interfaz gráfica pertinente, por lo que debíamos hacer un

ejercicio de intuición y mezclarlo con funciones programables que ahorrasen tiempo.

Por otro lado, el etiquetado de imágenes también ha sido una labora costosa de llevar a cabo, únicamente por la lentitud con la que se puede ejecutar esta tarea, por una parte, encontramos el problema de la gran cantidad de imágenes necesarias para la precisión requerida, y, por otra parte, la propia acción de etiquetar es bastante meticulosa, ya que debes ceñirte al objeto que deseas encuadrar.

6.3. Sprint 3

Este Sprint es uno de los más importantes de todo el proyecto, ya que contiene tareas que definen lo bien, o mal, que va a funcionar la aplicación. A continuación, se adjunta la parte del *Product Backlog* donde se definen las tareas a realizar.

Tabla 7. Descripción Sprint 3

Historia de usuario	Sprint	Descripción de la tarea	Prioridad	Puntos de historia
3	3	Entrenamiento personalizado del modelo de Inteligencia Artificial (IA)	M	40
3	3	Detección de vehículos a través de vídeos	C	8
3	3	Sistema de detección de elementos	M	5

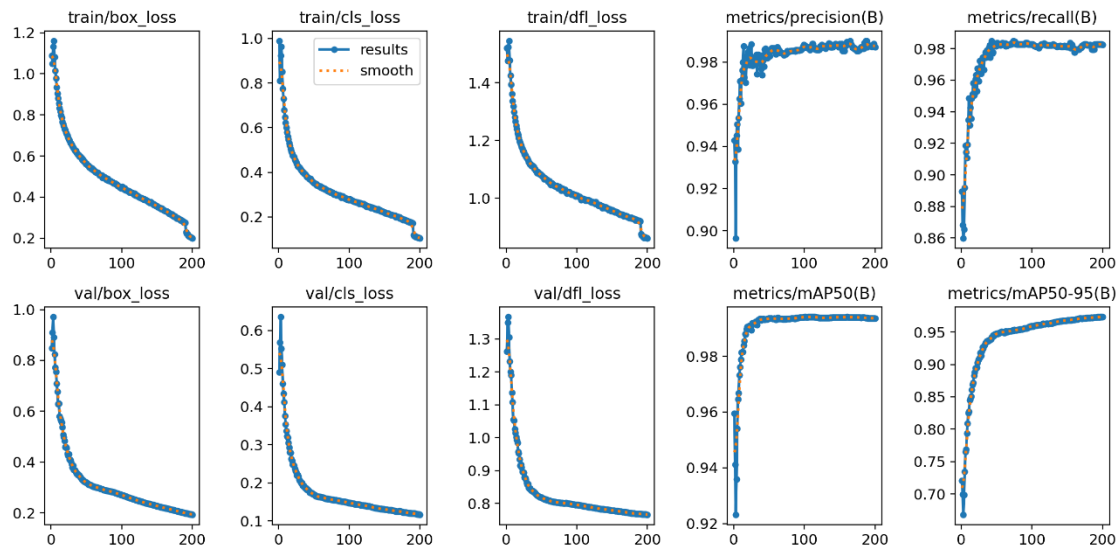
		basado en regiones		
--	--	-----------------------	--	--

Nota: elaboración propia.

6.3.1. Planificación del Sprint

Este Sprint se dedicará al entrenamiento y prueba del modelo personalizado de Inteligencia Artificial (IA). En primer lugar, se va a realizar el entrenamiento, para ello haremos uso del *dataset*, que previamente ha sido etiquetado, con ello, haremos un entrenamiento que dura un total de 72 horas, aproximadamente. Una vez finalizado el entrenamiento, disponemos del archivo que nos permite hacer uso de este modelo, y junto a él una serie de estadísticas, gráficas e imágenes que nos aportan información sobre cómo ha ido el proceso de entrenar.

Figura 5. Estadísticas de entrenamiento



Nota: imagen extraída del resultado de un entrenamiento de modelo personalizado de Inteligencia Artificial (IA), elaboración propia.

Tras ello se procede a la creación de un sistema que pruebe la exactitud del programa, este sistema consiste en la ejecución de un vídeo al que se le aplica este modelo, así podremos detectar, de forma visual, qué tal es su comportamiento.

Por último, implantaremos una detección por regiones. Aunque el programa actúe de forma genérica, para todo lo que detecta la cámara, lo que se pretende es conseguir un control sobre zonas concretas de los cruces. Estas regiones son dibujadas en pantalla, tienen forma rectangular, e implementan un contador que aumenta y disminuye en función de los elementos detectados, además permite modificar la ubicación de las zonas con un sistema que permite clicar y arrastrar por pantalla.

Figura 6. *Detección por regiones*



Nota: detección de vehículos en circulación dentro de regiones, elaboración propia.

6.3.2. Planificación del Sprint

La etapa del entrenamiento del modelo de Inteligencia Artificial (IA) ha sido la que más tiempo y trabajo ha llevado, el hecho de ejecutar un entrenamiento de este tipo hace que se sobrecargue el ordenador, lo que ha provocado una sobrecarga de la tasa de uso de los recursos, haciendo que

fuese, prácticamente imposible, usar el ordenador, mientras el entrenamiento estaba en ejecución.

Además, esta etapa, ha tenido que ser repetida en varias ocasiones, ya que, o no se conseguían los resultados esperados, la calidad de los datos no era del todo buena o simplemente se paraba por fallos. A pesar de todo, se consiguió encontrar un entrenamiento que satisficiera y resolviera con éxito los requisitos necesarios para poder realizar una aplicación de calidad.

En cuanto al sistema de detección por regiones, hubo problemas con la detección, ya que, a pesar de encontrar distintos elementos en su interior, no todos eran detectados, y por otro lado el contador no marcaba la cantidad correcta, siempre se incrementaba, pero no siempre disminuía. Todo esto ha sido satisfactoriamente solucionado, las regiones contabilizan el centro del elemento detectado, y en caso de que este pase las paredes de las regiones, se añadirá, o eliminará de la lista del contador.

6.4. Sprint 4

Este *Sprint* está dedicado a la parte del *backend* de la aplicación, generaremos el sistema de gestión de prioridades basado en umbrales, el cual determinará el orden de paso de los vehículos, y se realizará una mejora en el sistema de detección que permitirá conectar el modelo de Inteligencia Artificial (IA) con el teléfono móvil y se realizará la configuración de la API de TraCI.

Tabla 8. Descripción Sprint 4

Historia de usuario	Sprint	Descripción de la tarea	Prioridad	Puntos de historia
6	4	Sistema basado en umbrales para optimizar la circulación	M	8

8	4	Conexión entre el modelo entrenado y la cámara del móvil	M	5
7	4	Configurar la API de TraCI	M	8

Nota: elaboración propia.

6.4.1. Planificación del Sprint

Puesto que ya disponemos de un modelo entrenado que cumple los requerimientos para que la aplicación funcione de manera adecuada, es hora de generar una configuración para la regulación de este.

La primera tarea que vamos a desarrollar es la gestión de priorización de paso por la vía, haciendo uso de umbrales de preferencia. Por defecto, hemos desarrollado nuestro programa para que, en caso de no detectar ningún vehículo en circulación, mantenga las luces del cruce en color rojo, en el momento que algún vehículo entra en cualquiera de las regiones, el sistema le ofrece un paso de preferencia, dejando al resto de coches parados.

En el caso de encontrar un entorno donde la afluencia de vehículos sea elevada, se diseñó un sistema basado en umbrales, por defecto se calcula un tiempo ofrecido para que el usuario pase, y al superar el umbral, comienza a dársele prioridad a dicho grupo para facilitar la descongestión y evitar atascos.

Otra de las características implementadas, es la conexión entre el teléfono móvil y el ordenador, de esta forma, lo que se pretende conseguir, es simular una cámara de tráfico real. El sistema recoge las imágenes captadas por el móvil, y de forma instantánea, son enviadas al ordenador, el cual le aplica de forma directa el modelo de detección.

Por último, configuramos la API de TraCI, esta nos permite hacer una comunicación en tiempo real entre el simulador SUMO, y nuestro *backend*, de esta forma, obtenemos la posibilidad de cambiar cualquier elemento de forma instantánea a través del código, mientras la simulación se encuentra en marcha.

6.4.2. Retrospectiva del Sprint

Con el balance final de este *Sprint*, podemos determinar que ha sido positivo, tanto la integración del modelo personalizado con la cámara móvil, y la configuración y pruebas de la API de TraCI, han salido como se había planeado.

En cambio, el sistema de gestión basado de umbrales ha generado una serie de problemas que han acaparado gran parte del tiempo de desarrollo. El hecho de disponer de un sistema que se actualice de forma constante implica que ciertos efectos no se puedan llevar a cabo en su totalidad, el momento en el que se ajusta el tiempo de paso en una intersección, está basado por los datos reales de ese momento, en el instante que estos cambian, es posible que también cambie la preferencia de ese instante, por lo que, realmente, nunca se termina de completar el ciclo completo de tiempo que se le había asignado al semáforo.

Para solucionar esto, se ha implantado un sistema que mide el tiempo mínimo que debe pasar, hasta que se considere que debe de volver a poder actualizar el umbral, y en el caso que fuese necesario modificar el estado de los semáforos.

6.5. Sprint 5

Nos encontramos frente al último *Sprint*, es por lo que es el que menos tareas tiene, de esta forma se intenta asegurar un flujo de trabajo constante, concreto y con carácter a finalizar el proyecto. A continuación, podemos encontrar las tareas del *Product Backlog*.

Tabla 9. Descripción Sprint 5

Historia de usuario	Sprint	Descripción de la tarea	Prioridad	Puntos de historia
8	5	Unión entre el programa base, la API y	M	20

		el modelo entrenado		
6	5	Implantación de <i>multithreading</i> en la ejecución del programa	S	13

Nota: elaboración propia.

6.5.1. Planificación del Sprint

Tras disponer de un modelo de Inteligencia Artificial (IA) personalizado, y la configuración completa de un sistema de simulación que es capaz de transmitir los cambios en los parámetros de forma instantánea, mientras este se encuentra en ejecución, llega la hora de realizar la fusión entre ambos.

Para poder lograr un circuito cerrado capaz de detectar las imágenes en tiempo real, procesarlas a través de Inteligencia Artificial (IA), será necesario analizar esos datos, procesarlos, y realizar los cambios en la simulación en base a los cálculos establecidos, se ha tenido que generar un programa en el *backend* que trabaje de conductor entre estas tecnologías e interfiera entre ellas a través de funciones, lo que nos lleva directamente a la segunda tarea.

El hecho de trabajar entre distintas tecnologías que combinan y comparten datos entre sí, ha hecho que se decida trabajar con una librería *multithreading*, con ella, aprovechamos los hilos del procesador para que cada uno se encargue de realizar unas funciones sin tener que interrumpir, o esperar a la otra parte del programa. El objetivo de esta implementación es mantener siempre la detección en tiempo real, así nos aseguramos de que siempre está la detección disponible y actualizada.

6.5.2. Retrospectiva del Sprint

Este *Sprint* ha sido, sin ningún lugar a dudas, el que más problemática ha ocasionado. En general, la dinámica de ambas tareas es la de trabajar y

coordinar con los datos de diferentes fuentes, y esto nos ha llevado a problemáticas como, dependencias de ejecuciones o pérdida de datos.

El hecho de combinar la API de TraCI con el modelo de Inteligencia Artificial (IA), ha hecho que la detección del modelo llegue a depender de la ejecución de la simulación, es decir, en el momento que la simulación se paraba, la detección también, o si esta reducía la velocidad, el modelo también captaba a una velocidad más lenta. La parte positiva, es que ya se tenía en mente la implantación del *multithreading*, y este nos soluciona ese problema, ya que, aunque se comparten datos, la ejecución de ambas partes se hace de forma independiente.

El otro problema que se dio en el desarrollo de este *Sprint* fue la pérdida de datos. Tras la incorporación del *multithreading*, cada función disponía de una versión propia de las variables, incluso a la hora de pasarlas por parámetro. Había errores de acceso que hacían que los valores no se actualizaran o leyeran de forma correcta, incluido errores de acceso múltiple. Para solucionar este problema, se globalizaron las variables, haciendo que fueran accesibles por todas las funciones, en cualquier momento, y se creó un sistema de bloqueo de lectura y escritura, que aseguraba que cualquier modificación del dato fuese fiel a los valores reales.

7. DESPLIEGUE Y PRUEBA DE LA SOLUCIÓN

En este apartado abordaremos el plan de pruebas que se ha planteado, estudiado y abordado para la aplicación desarrollada en este trabajo. Realizaremos un recorrido en profundidad por el plan de pruebas, y después se hará de una revisión de su desarrollo y escalabilidad.

7.1. Plan de pruebas

La metodología ágil de Scrum se caracteriza por la implementación de testear cada módulo tras la finalización de este, esto nos permite asegurar la calidad de los distintos módulos que componen la aplicación, de esta forma nos aseguramos de construir un proyecto sobre una base sólida que garantiza el cumplimiento de los requisitos solicitados por el cliente.

En este apartado vamos a presentar el plan de pruebas generado, este plan, está organizado por Sprint, e historia de usuario correspondiente, además agregaremos una columna donde indicaremos el resultado de la prueba ejecutada, coloreando en color verde si la prueba ha salido de forma satisfactoria, y coloreando en color rojo si el resultado de la prueba no es satisfactorio.

Tabla 10. Descripción del plan de pruebas

ID	Sprint	Historia	Caso de prueba	Resultado
CP1-001	1	1	Comprobar la correcta ejecución del entorno	
CP1-002	1	2	Comprobar la correcta ejecución del entorno	

CP1-003	1	3	Comprobar la correcta ejecución del entorno	
CP2-001	2	1	Verificar dependencias e instalaciones	
CP2-001	2	3	Testear simulación de prueba	
CP2-002	2	4	Comprobar la generación automática de mapas	
CP2-003	2	5	Testear etiquetado de imágenes	
CP3-001	3	1	Comprobar entrenamiento del modelo	
CP3-002	3	2	Testear detección del modelo en vídeos	
CP3-003	3	3	Comprobar detección de regiones	

CP4-001	4	1	Verificar sistema basado en umbrales	
CP4-002	4	2	Testear detección a través de cámara	
CP4-003	4	3	Comprobar la modificación de datos en tiempo real a través de la API de TraCI	
CP5-001	5	1	Verificar la transmisión de datos en circuito cerrado	
CP5-002	5	2	Verificar la integridad de los datos usando multithreading	

Nota: breve descripción de la prueba a realizar y resultado obtenido, elaboración propia.

Una vez hemos establecido una visión general sobre los casos de prueba, podemos presentar la especificación de cada uno de ellos.

Tabla 11. Especificación de los casos de prueba

CP1-001	Comprobar la correcta ejecución del entorno
Historia de usuario	1
Descripción	Generar un programa base, en idioma Python que asegure que el entorno esté configurado
Precondiciones	Deben estar instalados todos los plugins que se van a utilizar y utilizar la versión de Python correspondiente al proyecto.
Resultado esperado	Este programa debe poder ejecutarse rápidamente mediante la versión establecida de Python
Errores	
CP1-002	Comprobar la correcta ejecución del entorno
Historia de usuario	2
Descripción	Creación de una carretera básica que garantice la generación de archivos de configuración
Precondiciones	
Resultado esperado	Generación de los archivos de configuración, almacenados en formato .xml
Errores	

CP1-003	Comprobar la correcta ejecución del entorno
Historia de usuario	3
Descripción	Ejecutar un archivo que albergue la configuración de una simulación
Precondiciones	Haber creado dicho archivo con NetEdit
Resultado esperado	El simulador SUMO es capaz de ejecutar y representar la simulación
Errores	
CP2-001	Verificación de la instalación de paquetes y dependencias
Historia de usuario	1
Descripción	Crear, compilar y ejecutar un código que haga uso de las librerías y paquetes
Precondiciones	Haber realizado la instalación de todas las dependencias del proyecto
Resultado esperado	Mostrar por pantalla la ejecución de una función específica de cada librería
Errores	En algún caso se han instalado versiones no compatibles u obsoletas
CP2-002	Comprobar simulación compleja de prueba
Historia de usuario	3

Descripción	Generar una red compleja, con flujos de circulación
Precondiciones	Debe incluir todos los elementos que intervendrán en la versión final de la red
Resultado esperado	Una ejecución de la simulación en SUMO, de unos vehículos que completan un circuito y son regulados por semáforos
Errores	
CP2-003	Comprobar la generación de mapas por código
Historia de usuario	4
Descripción	Generar diversas configuraciones de redes de circulación complejas, a través de código
Precondiciones	Estructurar y crear la disposición de archivos .xml necesarios
Resultado esperado	La generación de distintos casos de uso, que tengan distintas características que favorezcan las pruebas con la IA
Errores	Mal acceso a los archivos, y error en la escritura
CP2-004	Comprobación del etiquetado de las imágenes
Historia de usuario	5

Descripción	Revisar si el etiquetado de un conjunto de imágenes es compatible con el modelo que las va a entrenar
Precondiciones	Las imágenes deben haber sido etiquetadas antes de la comprobación, y exportadas a un directorio
Resultado esperado	Cada imagen debe disponer de un archivo de texto, que corresponde con su etiqueta, más la propia imagen
Errores	Error por mal encuadre en el etiquetado
CP3-001	Testear el modelo entrenado
Historia de usuario	1
Descripción	Someter a pruebas el modelo personalizado de IA
Precondiciones	El modelo tiene que haber sido entrenado con una configuración que permita reconocer vehículos, basándose en el dataset proporcionado
Resultado esperado	Correcta detección de vehículos en imágenes
Errores	Bajo nivel de precisión en la detección de los primeros entrenamientos
CP3-002	Comprobar la detección en vídeos

Historia de usuario	2
Descripción	Detección de vehículos en un vídeo donde haya vehículos en circulación
Precondiciones	El modelo debe ser capaz de detectar con alta precisión imágenes
Resultado esperado	El modelo debe captar y seguir el vehículo de manera precisa mientras este se mueve
Errores	Error por mal uso de las librerías que ejecutan los vídeos
CP3-003	Verificar la detección dentro de las regiones
Historia de usuario	3
Descripción	Detectar vehículos dentro de las regiones dibujadas en pantalla
Precondiciones	El modelo debe ser capaz de detectar vehículos en movimiento
Resultado esperado	La región en la que un vehículo entra debe incrementar su contador de vehículos
Errores	Error en el contador, en un inicio solo contaba aquellos que entraban
CP4-001	Verificar sistema basado en umbrales
Historia de usuario	1

Descripción	Aplicar un sistema, basado en umbrales, que gestione de manera eficiente la circulación del tráfico
Precondiciones	Debe recibir el estado actual de las regiones y los vehículos
Resultado esperado	Generación de un tiempo de fase de semáforo óptimo para la fluencia de tráfico de ese momento
Errores	Problemas con la transmisión de los datos, y solapamiento de tiempos
CP4-002	Comprobar la detección a través de cámara de vídeo
Historia de usuario	2
Descripción	Aplicar el modelo de IA a las imágenes en directo transmitidas desde el móvil al ordenador
Precondiciones	El backend debe detectar el móvil como webcam
Resultado esperado	Visionado y reconocimiento de vehículos instantáneo de las imágenes captadas por el móvil
Errores	El programa no detectaba la cámara del móvil por un conflicto de librerías
CP4-003	Testear la modificación de datos en tiempo real con la API de TraCI
Historia de usuario	3

Descripción	Ejecutar una simulación y modificar sus parámetros mientras esta se encuentra activa
Precondiciones	Iniciar la simulación con la conexión de TraCI
Resultado esperado	Un ejemplo simulado con un final preestablecido, que es modificado en tiempo real por el backend
Errores	Problemas de conexión con los puertos de la API
CP5-001	Comprobar la transmisión de datos en circuito cerrado
Historia de usuario	1
Descripción	Comprobar que es posible generar todo el circuito de captación, procesamiento y modificación de los parámetros del programa en base al reconocimiento de la IA
Precondiciones	Que todos los sistemas se encuentren vinculados por el backend
Resultado esperado	Un ejemplo de simulación que se ve modificado en tiempo real, por la detección de la IA, que envía los datos al backend, el cual, genera una nueva fase de regulación que se aplica con la API de TraCI a la simulación

Errores	Solapamiento con la ejecución entre distintas tecnologías, concretamente la API y el procesamiento de imagen de la IA
CP5-002	Verificar la integridad de los datos usando multithreading
Historia de usuario	2
Descripción	Comprobar la integridad e independización de los datos cuando son modificados por el backend
Precondiciones	Haber instaurado la tecnología multithreading a las funciones y variables pertinentes
Resultado esperado	La correcta lectura, modificación y envío de los datos
Errores	Pérdidas de datos en modificaciones y lecturas

Nota: especificación de las acciones

7.2. Escalabilidad, extensión, mantenimiento y soporte

La forma en la que se ha abordado este proyecto permite que este tenga unos altos índices de escalabilidad. En cuanto a la parte del *backend*, ha sido programada priorizando un sistema modular modularidad, de modo que cada función esta delimitada para una acción concreta, esto permite que sea fácil la ampliación de esta, de hecho, la personalización del proyecto se podría considerar infinita gracias a la generación de redes de circulación realizada a partir de código.

El modelo de Inteligencia Artificial (IA) personalizado, es totalmente ampliable, ya que, bastaría con incrementar el *dataset* del cual se nutre para

realizar el entrenamiento, y con ello, obtendríamos una ampliación de rango de alcance hacia más tipos de vehículos y elementos que podemos encontrar en la circulación.

El sistema basado en umbrales podría ampliarse para que no solo generase soluciones para situaciones actuales, si no que este, fuese capaz de generar flujos de circulación, de esta manera obtendríamos patrones de comportamiento que ayudarían a regular el tráfico. Por otro lado, se podría valorar la fusión de los distintos servicios de manera que todos compartiesen los datos, teniendo una información completa sobre la circulación de la ciudad en la que se encuentra instalado este sistema.

En cuanto al mantenimiento de la aplicación, considero que gira en torno a la actualización constante del modelo que actualiza las fases de priorización de tráfico y del modelo de detección, ya que son el motor de esta aplicación. Hoy en día estamos en constante evolución tecnológica, por lo que mantener esto actualizado hará que mantengamos la seguridad y precisión del sistema general.

Por último, queda por añadir la posibilidad de ampliación del *hardware*, ya que si se añaden a este sistema otro tipo de elementos que ayuden a captar información, como los sensores de detección o dispositivos IoT (Internet de las Cosas), podríamos ampliar este sistema de optimización de circulación fuera del ámbito de los cruces con semáforos, y adaptarlo a una mayor variedad de escenarios y necesidades.

8. CONCLUSIONES

Para finalizar este gran proyecto, es necesario y conveniente analizar los objetivos marcados al principio, y observar los que hemos conseguido alcanzar, haremos una valoración crítica de las partes que nos ayuden a clarificar si alcanzamos nuestros planteamientos.

8.1. Objetivos alcanzados

Al inicio de este proyecto fueron nombrados y descritos, una serie de objetivos que se pretendían alcanzar para poder elaborar un programa completo, eficiente y útil. Estos objetivos se dividían en dos partes, generales y específicos, a continuación, determinaremos si se ha logrado alcanzar cada uno de ellos.

- **OG-01.** Diseñar una aplicación de gestión de tráfico eficiente.

Como se ha ido describiendo a lo largo de todo este proyecto, se ha conseguido desarrollar una aplicación que es capaz de gestionar la circulación del tráfico, mejorando la seguridad y reduciendo las emisiones y tiempos de espera dentro de la vía, todo ello ha sido posible por la ayuda de la Inteligencia Artificial (IA), lo que nos ha llevado a conseguir un programa competente en la detección y regulación del tráfico en tiempo real.

- **OE-01.** Optimizar la circulación en situación de aglomeración de vehículos.

Como se ha comentado en el punto anterior, hemos conseguido crear una aplicación capaz de regular el tráfico de forma efectiva. El desarrollo de este objetivo específico ha sido un complemento que ha ayudado a mejorar el sistema general, ya que, haciendo uso de la detección en tiempo real de los vehículos, y gracias al sistema de umbrales que se ha desarrollado, hemos conseguido que los casos de mayor fluctuación de tráfico queden libres de tráfico en mucho menos tiempo.

- **OE-02.** Desarrollar la detección de situaciones de emergencia.

Este objetivo específico era el que más se alejaba de nuestra aplicación, no porque no esté relacionado, sino porque no podemos considerarlo como una

función básica en la que todo el mundo pensaría una vez se plantea este proyecto. Para el desarrollo de este objetivo, el cual no hemos logrado implementar, habría sido necesario un entrenamiento específico del modelo para detectar situaciones de emergencia, y en un caso óptimo, la resolución de este con un comunicado hacia los servicios correspondientes bien sea policía, ambulancia o bomberos.

8.2. Conclusiones del trabajo y personales

Con la finalización de este Trabajo de Fin de Grado (TFG) he podido observar todas aquellas motivaciones que me guiaron e incentivaron a investigar este tema. La mejora de la circulación en la vía urbana ha sido un acierto, ya que hoy es un campo aún muy verde y con grandes posibilidades para mejorar la seguridad, rendimiento y optimización.

El principal motivo por el que decidí embarcarme en esta temática, son los hechos cotidianos en la carretera, con los que vivimos en el día a día, y que todos pensamos que son innecesarios, como pueden ser esas esperas eternas en semáforos vacíos.

A pesar de disponer de ciertas ventajas que alivian un poco la fluidez del tráfico, como los semáforos que cambian de color cuando te detienes sobre el dibujo que hay en vía, son sistemas que podemos mejorar y llevarlos a otro nivel con las tecnologías que se desarrollan día a día. Con estas mejoras, no solo mejoramos la fluidez, sino la seguridad, el control y la reducción de huella en carretera, tanto a nivel de desgaste como de contaminación del medio ambiente.

Por otro lado, cabe destacar, que otra gran motivación que tuve a la hora de elegir este proyecto, fue el hecho de aplicar una tecnología que está en pleno auge del desarrollo como es la Inteligencia Artificial (IA), aplicada a una problemática de la vida cotidiana. El hecho de poder investigar por cuenta propia tecnologías, aún en desarrollo, hace que siempre surjan nuevas oportunidades e ideas, que desembocan en proyectos interesantes y útiles.

La elección y empleo de una metodología ágil me ha llevado a ser más consciente de cómo es trabajar en un proyecto real, fuera del ámbito de

educativo, aportándome grandes destrezas a nivel organizativo, en las tomas de decisiones y mi capacidad de priorización, con respecto al inicio de este proyecto.

Aunque haya habido frustración por falta de equipo, conocimiento y soltura, he podido desarrollar y adquirir habilidades generales en todo el trabajo, lo que me ha dado la confianza y certeza para saber que, con un mayor equipo, el proyecto habría obtenido mejores resultados.

8.3. Vías futuras

Tras la realización del proyecto en su totalidad, he sido consciente del grado de mejora y evolución que puede tomar este trabajo. Estas mejoras aportan valor, y abren nuevos caminos hacia la investigación de nuevos campos.

Antes de comentar las mejoras, quiero destacar que el ámbito de la Inteligencia Artificial (IA) es un campo en continuo cambio, y posiblemente el abanico de posibilidades se expanda con el tiempo.

Respecto a mejoras relacionadas con funcionalidades ya existentes, se podría ampliar la gama de detección de objetos, ampliándola a la totalidad de individuos que pasen por la vía, ya sean vehículos de transporte público, peatones, etc. Otro aspecto para considerar es la regulación del transporte público, ya que este afecta de forma directa a nuestro caso, del mismo modo que los peatones, pese a que ambos tengan vías de paso prioritarias para estos componentes, como son los pasos de peatones o vías de circulación exclusivas para autobuses y taxis, podría establecerse un sistema que detecte casos de preferencia donde se les tenga en cuenta.

La implementación que consideró más valor a la aplicación sería la detección de casos de emergencia, ya que, dado el caso donde no se puede prevenir un accidente, sería una forma óptima de localizarlo inmediatamente para poder acudir a él. Además, se podría hacer un estudio sobre el por qué ese accidente se ha provocado y aportar las medidas necesarias para que no vuelva a suceder.

Por último, hacer *hincapié* en la posibilidad de implementar un sistema que tome las lecturas en tiempo real, y en base a los datos, genere patrones de circulación según el flujo del día a día, lo que nos beneficiaría generando casos más específicos de regulación de tráfico, ya que, tendríamos la capacidad de aplicar una metodología de descongestión en base al caso que está por suceder.

9. BIBLIOGRAFÍA

- Trigas Gallego, M. (2024). *Desarrollo detallado de la fase de aprobación de un proyecto informático mediante el uso de metodologías ágiles*. [TFC]. Consultora: Ana Cristina Domingo Troncho. <http://hdl.handle.net/10609/17885>
- Ortiz Tanori, A. R., & García Mireles, G. A. (s.f.). *Planning poker*. XXXIII Semana Nacional de Investigación y Docencia en Matemáticas, Universidad de Sonora. <https://semana.mat.uson.mx/semanaxxxiii/cartel/pdf/CARTELAlanRaulOrtizTanori.pdf>
- Audi networks with traffic lights in the USA. (2016). Audi MediaCenter. <https://www.audi-mediacycenter.com/en/press-releases/audi-networks-with-traffic-lights-in-the-usa-7147>
- AWS. (s. f.). ¿Qué es una API? - Explicación de interfaz de programación de aplicaciones - AWS. Amazon Web Services, Inc. Recuperado 19 de junio de 2024, de <https://aws.amazon.com/es/what-is/api/>
- Cadavid, A. N., Fernández, J. D., & Morales, J. (2013). Revisión de metodologías ágiles para el desarrollo de software. *Prospectiva*, 11(2), 30. <https://doi.org/10.15665/rp.v11i2.36>
- ESMARTCITY. (2016, junio 3). CITRAM, Centro de Innovación y Gestión de la Movilidad del Consorcio Regional de Transportes de Madrid. ESMARTCITY. <https://www.esmartcity.es/comunicaciones/citram-centro-innovacion-gestion-movilidad-consorcio-regional-transportes-madrid>
- González, C. (2018). En qué consiste el aprendizaje automático (machine learning) y qué está aportando a la Neurociencia Cognitiva. *Ciencia Cognitiva*. <https://www.cienciacognitiva.org/?p=1697>
- Green Light: Reduce Traffic Emissions with AI - Google Research. (s. f.). Green Light: Reduce Traffic Emissions with AI - Google Research. Recuperado 19 de junio de 2024, de <https://sites.research.google/greenlight/>

Green Wave Traffic | SWARCO. (s. f.). Recuperado 20 de junio de 2024, de <https://www.swarco.com/mobility-future/intelligent-transportation-systems/green-wave-traffic>

IBM. (2021, octubre 6). ¿Qué es la Inteligencia Artificial (IA)? | IBM. <https://www.ibm.com/es-es/topics/artificial-intelligence>

IBM. (2023, mayo 16). ¿Qué es Deep Learning? | IBM. <https://www.ibm.com/es-es/topics/deep-learning>

In|Sync – Rhythm Engineering. (s. f.). Recuperado 20 de junio de 2024, de <https://rhythmtraffic.com/insync/#DownloadCatalog>

León, A. R. S., Rodríguez, M. G., & Veitia, B. P. (2011). Crud—Pg. Revista Cubana de Ciencias Informáticas, 5(1), 1-8.

Milton Keynes to host major self-driving vehicle trial | Milton Keynes City Council. (2023). <https://www.milton-keynes.gov.uk/news/2023/milton-keynes-host-major-self-driving-vehicle-trial>

RapidFlow Surtrac Is Now Miovision Surtrac | Miovision. (s. f.). Recuperado 20 de junio de 2024, de <https://miovision.com/rapidflow/>

SUMO Documentation. (2024, ltima revisión). <https://sumo.dlr.de/docs/index.html>

Surtrac (Scalable URban TRAffic Control). (s. f.). US Ignite. Recuperado 20 de junio de 2024, de <https://www.us-ignite.org/tools/data-tool/surtrac-scalable-urban-traffic-control/>

Tarantola, A. (2023, octubre 10). Google's AI stoplight program is now calming traffic in a dozen cities worldwide. Engadget. <https://www.engadget.com/google-ai-stoplight-program-project-green-light-sustainability-traffic-110015328.html>

Tinoco Gómez, O., Rosales López, P. P., & Salas Bacalla, J. (2014). Criterios de selección de metodologías de desarrollo de software. Industrial Data, 13(2), 070. <https://doi.org/10.15381/idata.v13i2.6191>

TraCI - Documentación SUMO. (2024, ltima revisión).
<https://sumo.dlr.de/docs/TraCI.html>

Traffic signals | Milton Keynes City Council. (s. f.). Recuperado 20 de junio de 2024, de <https://www.milton-keynes.gov.uk/highways/street-lighting-and-traffic-signals/traffic-signals>

